# ANNAMACHARYA UNIVERSITY

**(ESTD UNDER AP PRIVATE UNIVERSITIES (ESTABLISHMENT AND REGULATION)**
**(UNIVERSITY LISTED IN UGC AS PER THE SECTION 2(f) OF THE UGC ACT, 1956)**
**RAJAMPET, Annamayya District, AP – 516126,**

**Title of the Course:** Data Mining
**Category:** PE
**Year:** III B.Tech
**Semester:** I Semester
**Course Code:** 24ACSE53T
**Branch/es:** CSE,CSE(AIML) & CSE(IOTCSBCT)

| Lecture Hours | Tutorial Hours | Practice Hours | Credits | Total Hours |
|---|---|---|---|---|
| 3 | - | - | 3 | 54 |

**Course Objectives:**

1. Explain the fundamentals of data mining, data preprocessing techniques, and key issues in data mining.
2. Describe data warehousing concepts and association rule mining techniques for large datasets.
3. Apply classification and prediction techniques for analyzing data.
4. Analyze different clustering methods and outlier detection techniques for data analysis.
5. Evaluate the effectiveness of data mining techniques using the ORANGE tool on real-world datasets.

**Course Outcomes:**

Upon completion of the course, the students should be able to:

1. Describe data mining functionalities and perform basic data preprocessing operations.
2. Apply data warehouse models and association rule mining techniques to extract patterns.
3. Analyze and compare classification and prediction techniques based on performance.
4. Evaluate clustering algorithms and identify suitable methods for different data types.
5. Develop data mining models and infer knowledge from datasets using the ORANGE tool.

**Unit 1** **12**

Introduction: Fundamentals of data mining, Data Mining Functionalities, Classification of Data Mining systems, Data Mining Task Primitives, Major issues in Data Mining, Data Preprocessing: Needs Preprocessing the Data, Data Cleaning, Data Integration and Transformation, Data Reduction, Discretization and Concept Hierarchy Generation.

**Unit 2** **10**

Warehousing: Data Warehouse and OLAP Technology for Data Mining Data Warehouse, Multidimensional Data Model, Data Warehouse Architecture

Mining Association Rules In Large Databases: Basic Concepts and a Road Map, Efficient and Scalable Frequent Item set Mining Methods, Mining Various Kinds of Association Rules, From Association Mining to Correlation Analysis.

**Unit 3** **10**

Classification And Prediction: Issues Regarding Classification and Prediction, Classification by Decision Tree Induction, Bayesian Classification, Classification by Back propagation, Support Vector Machines, Other Classification Methods, Prediction, Classifier Accuracy.

**Unit 4** **10**

Cluster Analysis: Types of Data in Cluster Analysis, A Categorization of Major Clustering Methods, Partitioning Methods, Hierarchical Methods Density-Based Methods, Grid-Based Methods, Model- Based Clustering Methods, Outlier Analysis.

Unit 5 12

Orange Data Mining Tool: Introduction to Orange, Datasets – Introduction, Iris plants database, CSV File Import, Transform, Association Rule Mining, Decision Tree, KNN, Naïve Bayes, Linear Regression, Clustering algorithms.

**TEXT BOOK:**

1. Jiawei Han and Micheline Kamber, —Data Mining Concepts and Techniques, Third Edition, Elsevier, 2012.

**REFERENCES:**

1. Alex Berson and Stephen J.Smith, —Data Warehousing, Data Mining & OLAP¦, Tata McGraw – Hill Edition, 35th Reprint 2016.
2. K.P. Soman, Shyam Diwakar and V. Ajay, —Insight into Data Mining Theory and Practice, Eastern Economy Edition, Prentice Hall of India, 2006.
3. Ian H.Witten and Eibe Frank, —Data Mining: Practical Machine Learning Tools and Techniques, Elsevier, Second Edition.

CO-PO Mapping:

| Course Outcomes | Engineering Knowledge | Problem Analysis | Design/Development of solutions | Conduct investigations of complex problems | Engineering tool usage | The engineer and The world | Ethics | Individual and Collaborative team work | Communication | Project management and finance | Life-long learning | PSO1 | PSO2 | PSO3 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 24ACSE53T.1 | 3 | 2 | 1 | 1 | 2 | - | - | - | - | - | 1 | 2 | 1 | - |
| 24ACSE53T.2 | 3 | 3 | 2 | 2 | 3 | - | - | - | 1 | 1 | 2 | 3 | 2 | 1 |
| 24ACSE53T.3 | 3 | 3 | 2 | 3 | 2 | - | - | - | 1 | - | 2 | 3 | 2 | 2 |
| 24ACSE53T.4 | 3 | 3 | 2 | 3 | 2 | 1 | - | - | 1 | - | 3 | 3 | 2 | 2 |
| 24ACSE53T.5 | 3 | 3 | 3 | 3 | 3 | 1 | - | 1 | 2 | 1 | 3 | 3 | 3 | 3 |

**Unit 1**

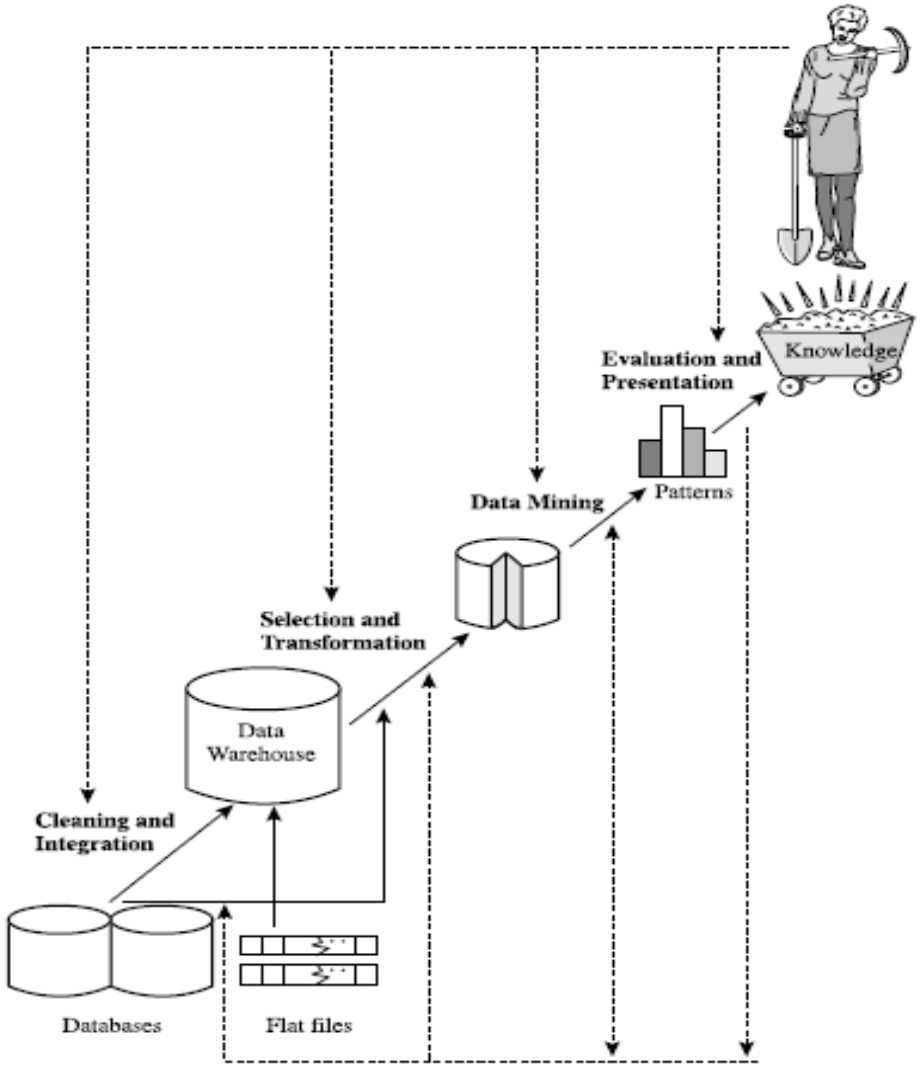
Introduction: Fundamentals of data mining, Data Mining Functionalities, Classification of Data Mining systems, Data Mining Task Primitives, Major issues in Data Mining, Data Preprocessing: Needs Preprocessing the Data, Data Cleaning, Data Integration and Transformation, Data Reduction, Discretization and Concept Hierarchy Generation.

## WHAT IS DATA MINING – DEFINITION

Data mining refers to extracting or mining" knowledge from large amounts of data. There are many other terms related to data mining, such as knowledge mining, knowledge extraction, data/pattern analysis, data archaeology, and data dredging.

Many people treat data mining as a synonym for another popularly used term, Knowledge Discovery in Databases", or KDD

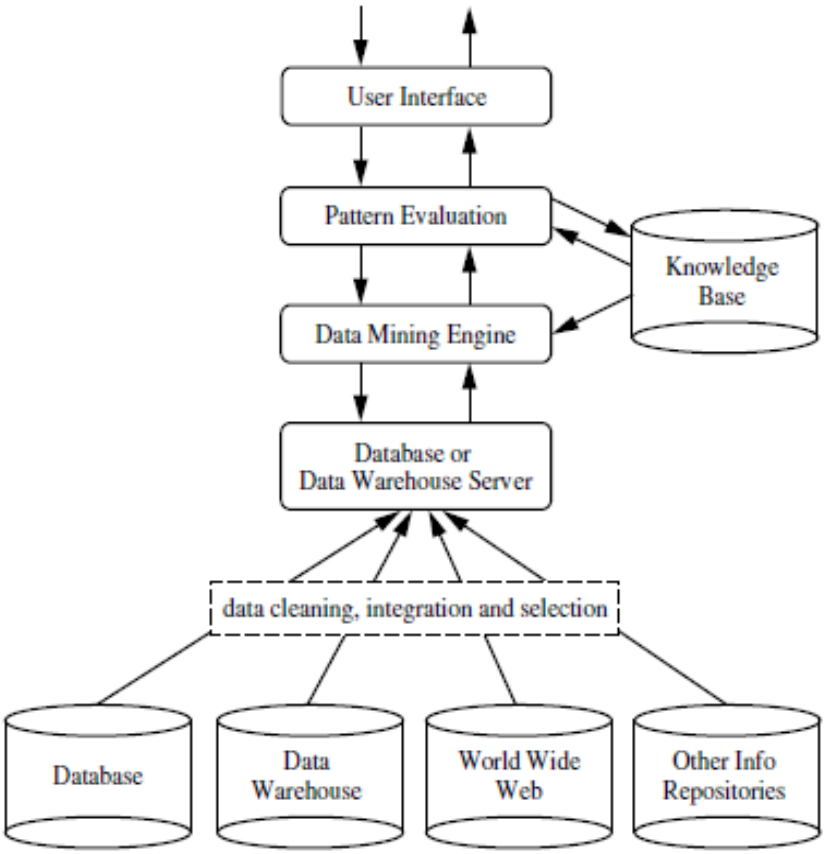
Knowledge discovery as a process is depicted in figure.



It consists of an iterative sequence of the following steps:

- **Data cleaning:** to remove noise or irrelevant data
- **Data integration:** where multiple data sources may be combined
- **Data selection:** where data relevant to the analysis task are retrieved from the database
- **Data transformation:** where data are transformed or consolidated into forms appropriate for mining by performing summary or aggregation operations
- **Data mining:** an essential process where intelligent methods are applied in order to extract data patterns
- **Pattern evaluation:** to identify the truly interesting patterns representing knowledge based on some interestingness measures
- **Knowledge presentation:** where visualization and knowledge representation techniques are used to present the mined knowledge to the user.

**Architecture of a typical data mining system/Major Components**




Data mining is the process of discovering interesting knowledge from large amounts of data stored either in databases, data warehouses, or other information repositories. Based on this view, the architecture of a typical data mining system may have the following major components:

1. **A database, data warehouse, or other information repository**, which consists of the set of databases, data warehouses, spreadsheets, or other kinds of information repositories containing the student and course information.
2. **A database or data warehouse server** which fetches the relevant data based on users 'data mining

requests.
3. **A knowledge base** that contains the domain knowledge used to guide the search or to evaluate the interestingness of resulting patterns. For example, the knowledge base may contain metadata which describes data from multiple heterogeneous sources.
4. **A data mining engine**, which consists of a set of functional modules for tasks such as classification, association, classification, cluster analysis, and evolution analysis.
5. **A pattern evaluation** module that works in tandem with the data mining modules by employing interestingness measures to help focus the search towards interestingness patterns.
6. **A graphical user interface** that allows the user an interactive approach to the data mining system. This module communicates between users and the data mining system.

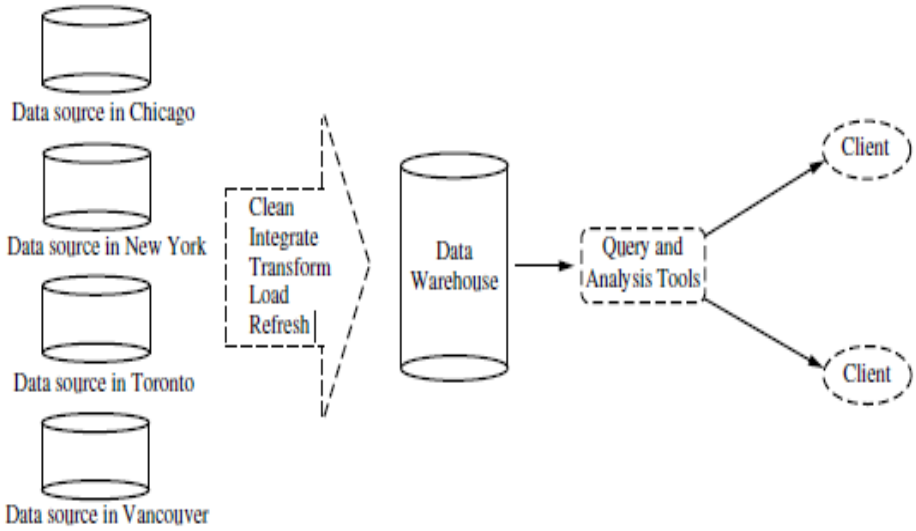
**KINDS OF DATA BASES**
Data mining should be applicable to any kind of data repository, as well as to transient data, such as data streams.
The scope of our examination of data repositories will include relational databases, data warehouses, transactional databases, advanced database systems, flat files, data streams, and the World Wide Web.

Advanced database systems include object-relational databases and specific application-oriented databases, such as spatial databases, time-series databases, text databases, and multimedia databases.

**Relational Databases**
A relational database is a collection of tables, each of which is assigned a unique name.
Each table consists of a set of attributes (columns or fields) and usually stores a large set of tuples (records or rows). Each tuple in a relational table represents an object identified by a unique key and described by a set of attribute values.
A semantic data model, such as an entity-relationship (ER) data model, is often constructed for relational databases.
Relational data can be accessed by database queries written in a relational query language, such as SQL

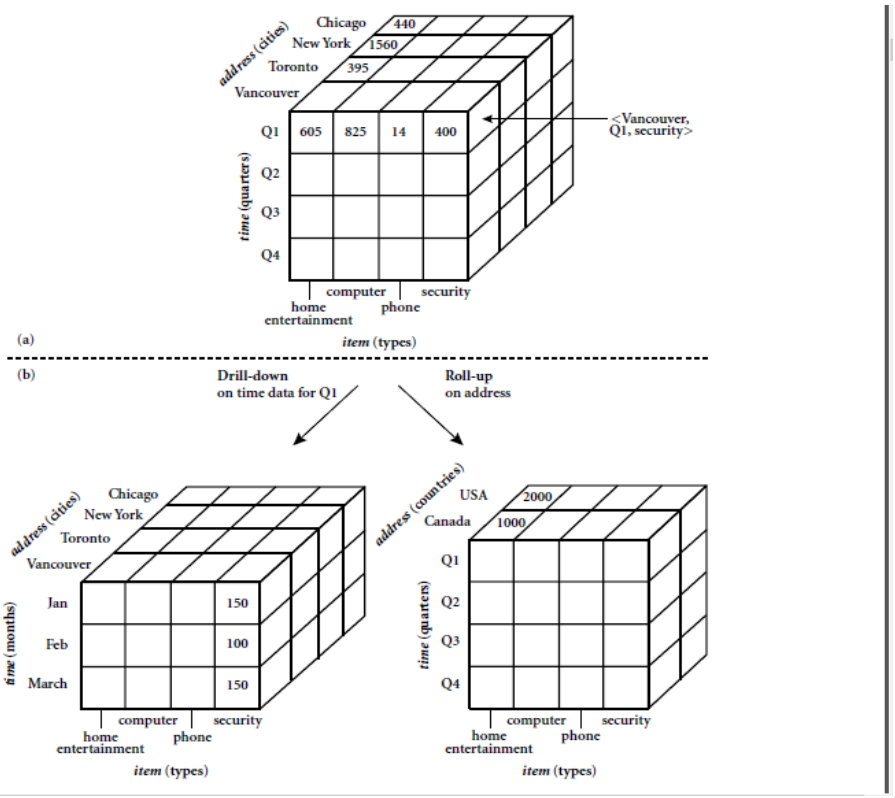
**DataWarehouses**
Suppose that AllElectronics is a successful international company, with branches around the world. Each branch has its own set of databases. The president of AllElectronics has asked you to provide an analysis of the company's sales per item type per branch for the third quarter. This is a difficult task, particularly since the relevant data are spread out over several databases, physically located at numerous sites.

If AllElectronics had a data warehouse, this task would be easy. A data warehouse is a repository of information collected from multiple sources, stored under a unified schema, and that usually resides at a single site. Data warehouses are constructed via a process of data cleaning, data integration, data transformation, data loading, and periodic data refreshing.




A data warehouse is usually modeled by a multidimensional database structure, where each dimension corresponds to an attribute or a set of attributes in the schema, and each cell stores the value of some aggregate measure, such as count or sales amount. The actual physical structure of a data warehouse may be a relational data store or a multidimensional data cube.

A data cube for AllElectronics. A data cube for summarized sales data of AllElectronics is presented in Figure.The cube has three dimensions: address (with city values Chicago, New York, Toronto, Vancouver), time (with quarter values Q1, Q2, Q3, Q4), and item(with itemtype values home entertainment, computer, phone, security).



**Transactional Databases**

A transactional database consists of a file where each record represents a transaction. A transaction typically includes a unique transaction identity number (trans ID) and a list of the items making up the transaction.

| *trans_ID* | *list of item_IDs* |
|---|---|
| T100 | I1, I3, I8, I16 |
| T200 | I2, I8 |
| ... | ... |

As an analyst of the AllElectronics database, you may ask, "Show me all the items purchased by Sandy Smith" or "How many transactions include item number I3?" Answering such queries may require a scan of the entire transactional database.

**Advanced Data and Information Systems and Advanced Applications**

**Object-Relational Databases**

Object-relational databases are constructed based on an object-relational data model.

This model extends the relational model by providing a rich data type for handling complex objects and object orientation.

**Each object has associated with it the following:**

- A set of variables that describe the objects. These correspond to attributes in the entity-relationship and relational models.
- A set of messages that the object can use to communicate with other objects, or with the rest of the database system.
- A set of methods, where each method holds the code to implement a message. Upon receiving a message, the method returns a value in response.

**Temporal Databases, Sequence Databases, and Time-Series Databases**

- A temporal database typically stores relational data that include time-related attributes. These attributes may involve several timestamps, each having different semantics.
- A sequence database stores sequences of ordered events, with or without a concrete notion of time. Examples include customer shopping sequences, Web click streams, and biological sequences.
- A time-series database stores sequences of values or events obtained over repeated measurements of time (e.g., hourly, daily, weekly).

**Spatial Databases and Spatiotemporal Databases**

Spatial databases contain spatial-related information. Examples include geographic (map) databases, very large-scale integration (VLSI) or computed-aided design databases, and medical and satellite image databases.A spatial database that stores spatial objects that change with time is called a Spatio temporal database

**Text Databases and Multimedia Databases**

Text databases are databases that contain word descriptions for objects. These word descriptions are usually not simple keywords but rather long sentences or paragraphs, such as product specifications, error or bug reports, warning messages, summary reports, notes, or other documents.

Multimedia databases store image, audio, and video data.

**Heterogeneous Databases and Legacy Databases**

A heterogeneous database consists of a set of interconnected, autonomous component databases. The components communicate in order to exchange information and answer queries. Objects in one component database may differ greatly from objects in other component databases, making it difficult to assimilate their semantics into the overall heterogeneous database.

A legacy database is a group of heterogeneous databases that combines different kinds of data systems, such as relational or object-oriented databases, hierarchical databases, network databases, spreadsheets, multimedia databases, or file systems.

**Data Streams**

Many applications involve the generation and analysis of a new kind of data, called stream data, where data flow in and out of an observation platform (or window) dynamically. Such data streams have the following unique features: huge or possibly infinite volume, dynamically changing, flowing in and out in a fixed order, allowing only one or a small number of scans, and demanding fast (often real-time) response time. Typical examples of data streams include various kinds of scientific and engineering data, time-series data

**The World Wide Web**

The World Wide Web and its associated distributed information services, such as Yahoo!, Google, America Online, and AltaVista, provide rich, worldwide, on-line information services, where data objects are linked together to facilitate interactive access.

**Data Mining—What Kinds of Patterns Can Be Mined?**

Data mining functionalities are used to specify the kind of patterns to be found in data mining tasks. In general, data mining tasks can be classified into two categories:

1.Descriptive

2.Predictive.

Descriptive mining tasks characterize the general properties of the data in the database.

Predictive mining tasks perform inference on the current data in order to make predictions.

**Concept/Class Description: Characterization and Discrimination**

Data can be associated with classes or concepts.

For example,

in the AllElectronics store, classes of items for sale include computers and printers, and concepts of customers include bigSpenders and budget Spenders.

It can be useful to describe individual classes and concepts in summarized, concise, and yet precise terms. Such descriptions of a class or a concept are called class/concept descriptions.

These descriptions can be derived via

1. Data characterization
2. Data discrimination

**Data characterization**

is a summarization of the general characteristics or features of a target class of data. The data corresponding to the user-specified class are typically collected by a database query.

For example, to study the characteristics of software products whose sales increased by 10% in the last year, the data related to such products can be collected by executing an SQL query

The output of data characterization can be presented in various forms. Examples include pie charts, bar charts, curves, multidimensional data cubes, and multidimensional tables, including crosstabs.

**Data discrimination**

is a comparison of the general features of target class data objects with the general features of objects from one or a set of contrasting classes.

The target and contrasting classes can be specified by the user, and the corresponding data objects

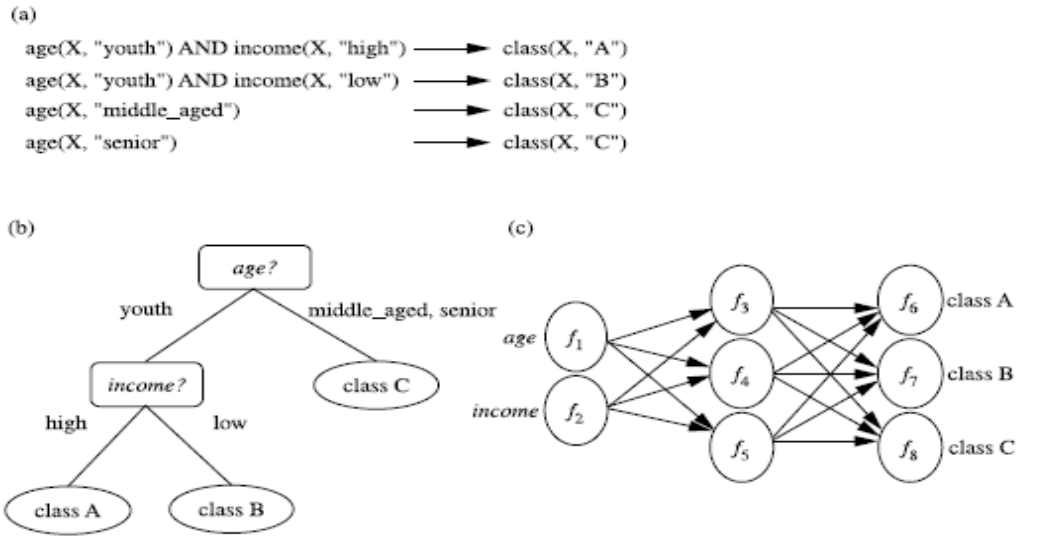
retrieved through database queries.
For example, the user may like to compare the general features of software products whose sales increased by 10% in the last year with those whose sales decreased by at least 30% during the same period. The methods used for data discrimination are similar to those used for data characterization.
Here also we use the same pie charts, bar charts, curves, multidimensional data cubes, and multidimensional tables, including crosstabs for output.

**Mining Frequent Patterns, Associations, and Correlations**
Frequent patterns, as the name suggests, are patterns that occur frequently in data. There are many kinds of frequent patterns, including itemsets, subsequences, and substructures.
A frequent itemset typically refers to a set of items that frequently appear together in a transactional data set, such as milk and bread.
A frequently occurring subsequence, such as the pattern that customers tend to purchase first a PC, followed by a digital camera, and then a memory card, is a (frequent) sequential pattern.
A substructure can refer to different structural forms, such as graphs, trees, or lattices, which may be combined with itemsets or subsequences. If a substructure occurs frequently, it is called a (frequent) structured pattern.

Eg:
Association analysis. Suppose, as a marketing manager of AllElectronics, you would like to determine which items are frequently purchased together within the same transactions.
An example of such a rule, mined from the AllElectronics transactional database, is
buys(X, “computer”)=>buys(X, “software”) [support = 1%; confidence = 50%] where X is a variable representing a customer. A confidence, or certainty, of 50% means that if a customer buys a computer, there is a 50% chance that she will buy software as well. A 1% support means that 1% of all of the transactions under analysis showed that computer and software were purchased together. This association rule involves a single attribute or predicate (i.e., buys) that repeats.
association rules are discarded as uninteresting if they do not satisfy both a minimum support threshold and a minimum confidence threshold.

**Classification and Prediction**
Classification is the process of finding a model (or function) that describes and distinguishes data classes or concepts, for the purpose of being able to use the model to predict the class of objects whose class label is unknown. The derived model is based on the analysis of a set of training data (i.e., data objects whose class label is known).
“How is the derived model presented?” The derived model may be represented in various forms, such as classification (IF-THEN) rules, decision trees, mathematical formulae, or neural networks.

A decision tree is a flow-chart-like tree structure, where each node denotes a test on an attribute value, each branch represents an outcome of the test, and tree leaves represent classes or class distributions. Decision trees can easily be converted to classification rules.
A neural network, when used for classification, is typically a collection of neuron-like processing units with weighted connections between the units.
There are many other methods for constructing classification models, such as naïve Bayesian classification, support vector machines, and k-nearest neighbor classification.

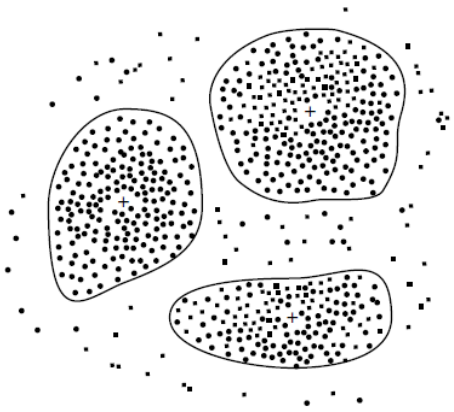



A classification model can be represented in various forms, such as (a) IF-THEN rules, (b) a decision tree, or a (c) neural network.

Eg:
Classification and prediction. Suppose, as sales manager of AllElectronics, you would like to classify a large set of items in the store, based on three kinds of responses to a sales campaign: good response, mild response, and no response. You would like to derive a model for each of these three classes based on the descriptive features of the items, such as price, brand, place made, type, and category.

**Cluster Analysis**

"What is cluster analysis?" Unlike classification and prediction, which analyze class-labeled data objects, clustering analyzes data objects without consulting a known class label.



The objects are clustered or grouped based on the principle of maximizing the intra class similarity and minimizing the interclass similarity. That is, clusters of objects are formed so that objects within a cluster have high similarity in comparison to one another, but are very dissimilar to objects in other clusters.
Eg:
Cluster analysis can be performed on AllElectronics customer data in order to identify homogeneous

subpopulations of customers. These clusters may represent individual target groups for marketing.

**Outlier Analysis**

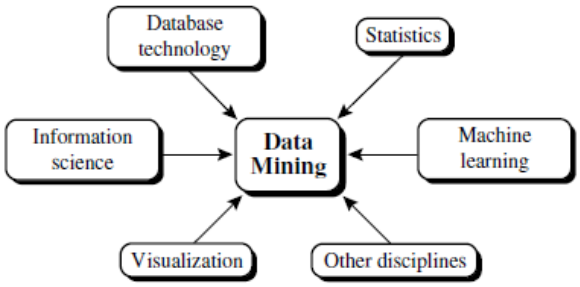
A database may contain data objects that do not comply with the general behavior or model of the data. These data objects are outliers. Most data mining methods discard outliers as noise or exceptions. However, in some applications such as fraud detection, the
rare events can be more interesting than the more regularly occurring ones. The analysis
of outlier data is referred to as outlier mining.
Outliers may be detected using statistical tests that assume a distribution or probability model for the data, or using distance measures where objects that are a substantial distance from any other cluster are considered outliers.
Outlier analysis. Outlier analysis may uncover fraudulent usage of credit cards by detecting purchases of extremely large amounts for a given account number in comparison to regular charges incurred by the same account. Outlier values may also be detected with respect to the location and type of purchase, or the purchase frequency.

Eg:
Outlier analysis. Outlier analysis may uncover fraudulent usage of credit cards by detecting purchases of extremely large amounts for a given account number in comparison to regular charges incurred by the same account. Outlier values may also be detected with respect to the location and type of purchase, or the purchase frequency.

**Evolution Analysis**

Data evolution analysis describes and models regularities or trends for objects whose behavior changes over time. Although this may include characterization, discrimination, association and correlation analysis, classification, prediction, or clustering of time related
data, distinct features of such an analysis include time-series data analysis, sequence or periodicity pattern matching, and similarity-based data analysis.
Eg:
Evolution analysis. Suppose that you have the major stock market (time-series) data of the last several years available from the New York Stock Exchange and you would like to invest in shares of high-tech industrial companies. A data mining study of stock exchange data may identify stock evolution regularities for overall stocks and for the stocks of particular companies. Such regularities may help predict future trends in stock market prices, contributing to your decision making regarding stock investments.

**Classification of Data Mining Systems**




Data mining is an interdisciplinary field, the confluence of a set of disciplines, including database

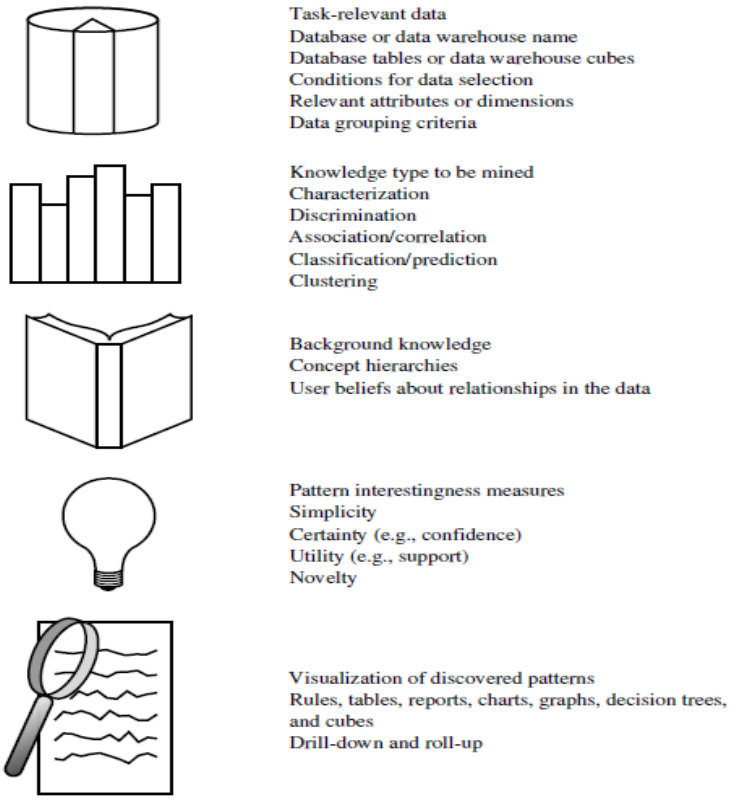
systems, statistics, machine learning, visualization, and information science.
Depending on the kinds of data to be mined or on the given data mining application, the data mining system may also integrate techniques from spatial data analysis, information retrieval, pattern recognition, image analysis, signal processing, computer graphics, Web technology, economics, business, bioinformatics, or psychology.

**Classification according to the kinds of databases mined:**
A data mining system can be classified according to the kinds of databases mined. Database systems can be classified according to different criteria (such as data models, or the types of data or applications involved), each of which may require its own data mining technique. Datamining systems can therefore be classified accordingly.
**Classification according to the kinds of knowledge mined:**
Data mining systems can be categorized according to the kinds of knowledge they mine, that is, based on data mining functionalities, such as characterization, discrimination, association and correlation analysis, classification, prediction, clustering, outlier analysis, and evolution analysis.
**Classification according to the kinds of techniques utilized:**
Data mining systems can be categorized according to the underlying data mining techniques employed. These techniques can be described according to the degree of user interaction involved (e.g., autonomous systems, interactive exploratory systems, query-driven systems) or the methods of data analysis employed (e.g., database-oriented or data warehouse–oriented techniques, machine learning, statistics, visualization, pattern recognition, neural networks, and so on).
**Classification according to the applications adapted:**
Data mining systems can also be categorized according to the applications they adapt. For example, data mining systems may be tailored specifically for finance, telecommunications, DNA, stock markets, e-mail, and so on.

**Data Mining Task Primitives**
Each user will have a data mining task in mind, that is, some form of data analysis that he or she would like to have performed. A data mining task can be specified in the form of a data mining query, which is input to the data mining system.
A data mining query is defined in terms of data mining task primitives. These primitives allow the user to interactively communicate with the data mining system during discovery in order to direct the mining process, or examine the findings from different angles or depths.

**The set of task-relevant data to be mined:** This specifies the portions of the database or the set of data in which the user is interested. This includes the database attributes or data warehouse dimensions of interest (referred to as the relevant attributes or dimensions).
**The kind of knowledge to be mined:** This specifies the data mining functions to be performed, such as characterization, discrimination, association or correlation analysis, classification, prediction, clustering, outlier analysis, or evolution analysis.
**The background knowledge to be used in the discovery process:** This knowledge about the domain to be mined is useful for guiding the knowledge discovery process and for evaluating the patterns found. Concept hierarchies are a popular form of background knowledge, which allow data to be mined at multiple levels of abstraction.
**Pattern interestingness measure:** This primitive allows users to specify functions that are used to separate uninteresting patterns from knowledge and may be used to guide the mining process, as well as to evaluate the discovered patterns. This allows the user to confine the number of uninteresting patterns returned by the process, as a data mining process may generate a large number of patterns. Interestingness measures can be specified for such pattern characteristics as simplicity, certainty, utility and novelty.
**The expected representation for visualizing the discovered patterns:** This refers to the form in which discovered patterns are to be displayed, which may include rules, tables, charts, graphs, decision trees,

and cubes.




**DATA PREPROCESSING**

Data preprocessing describes processing performed on raw data to prepare it for another processing procedure. Commonly used as a preliminary data mining practice, data preprocessing transforms the data into a format that will be more easily and effectively processed for the purpose of the user.

**Why Data Preprocessing?**

Data in the real world is dirty. It can be in incomplete, noisy and inconsistent from. These data needs to be preprocessed in order to help improve the quality of the data, and quality of the mining results.

If no quality data, then no quality mining results. The quality decision is always based on the quality data.

If there is much irrelevant and redundant information present or noisy and unreliable data, then knowledge discovery during the training phase is more difficult

Incomplete data: lacking attribute values, lacking certain attributes of interest, or containing only aggregate data.

e.g., occupation = 1.

Noisy data: containing errors or outliers data.

e.g., Salary = -100

Inconsistent data: containing discrepancies in codes or names.

e.g., Age = ―42 ‖ Birthday=03/07/1997

Incomplete data may come from

―Not applicable data value when collected

Different considerations between the time when the data was collected and when it is analyzed.

Human/hardware/software problems

Noisy data (incorrect values) may come from Faulty data collection by instruments Human or computer error at data entry Errors in data transmission Inconsistent data may come from Different data sources

**Major Tasks in Data Preprocessing**

• Data cleaning

Fill in missing values, smooth noisy data, identify or remove outliers, and resolve in consistencies

• Data integration

Integration of multiple databases, data cubes, or files

• Data transformation

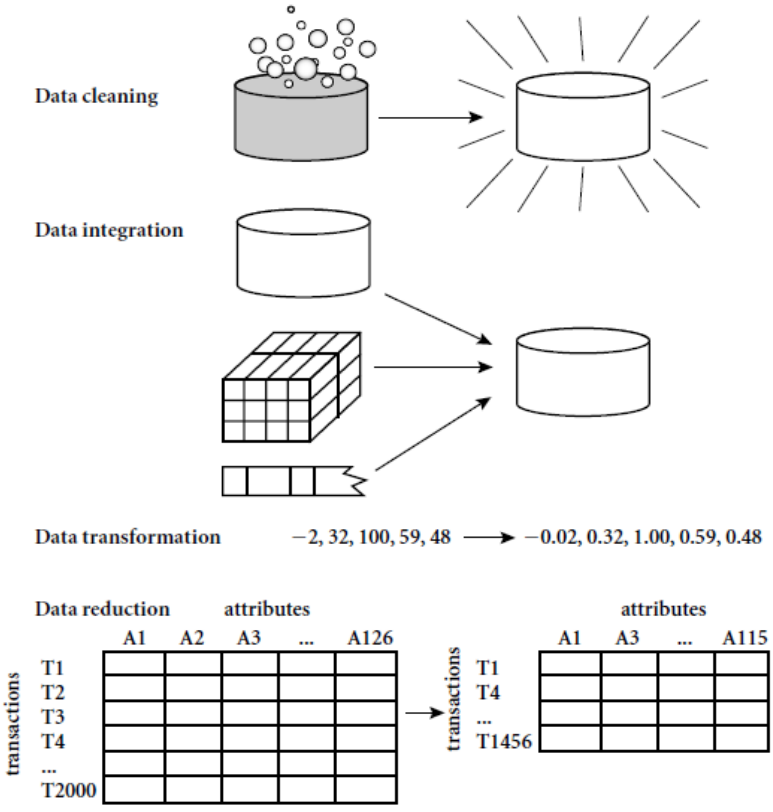
Normalization and aggregation

• Data reduction

Obtains reduced representation in volume but produces the same or similar analytical results

• Data discretization

Part of data reduction but with particular importance, especially for numerical data Forms of Data Preprocessing




**Data cleaning** (or data cleansing) routines attempt to fill in missing values, smooth out noise while identifying outliers, and correct inconsistencies in the data.

**Missing Values:**

Imagine that you need to analyze AllElectronics sales and customer data. You note that many tuples have no recorded value for several attributes, such as customer income. How can you go about filling in the missing values for this attribute? Let's look at the following methods:

1. Ignore the tuple: This is usually done when the class label is missing (assuming the mining task involves classification). This method is not very effective, unless the tuple contains several attributes with missing values. It is especially poor when the percentage of missing values per attribute varies considerably.

2. Fill in the missing value manually: In general, this approach is time-consuming and may not be feasible given a large data set with many missing values.

3. Use a global constant to fill in the missing value: Replace all missing attribute values by the same

constant, such as a label like "Unknown" or -¥. If missing values are replaced by, say, "Unknown," then the mining program may mistakenly think that they form an interesting concept, since they all have a value in common—that of "Unknown." Hence, although this method is simple, it is not fool proof.
4. Use the attribute mean to fill in the missing value: For example, suppose that the average income of AllElectronics customers is $56,000. Use this value to replace the missing value for income.
5. Use the attribute mean for all samples belonging to the same class as the given tuple:
For example, if classifying customers according to credit risk, replace the missing value with the average income value for customers in the same credit risk category as that of the given tuple.
6. Use the most probable value to fill in the missing value: This may be determined with regression, inference-based tools using a Bayesian formalism, or decision tree induction. For example, using the other customer attributes in your data set, you may construct a decision tree to predict the missing values for income.

**Noisy Data:**

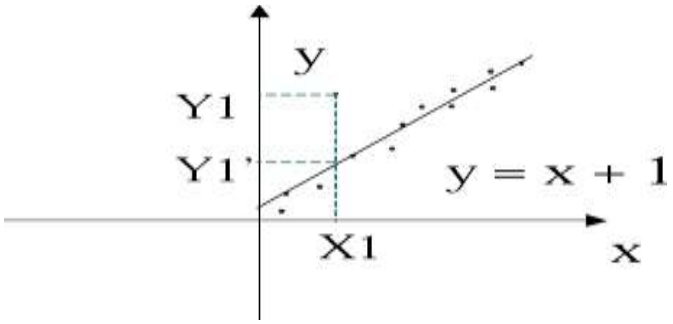
"What is noise?" Noise is a random error or variance in a measured variable. Given a numerical attribute such as, say, price, how can we "smooth" out the data to remove the noise?
Let's look at the following data smoothing techniques:
Sorted data for price (in dollars): 4, 8, 15, 21, 21, 24, 25, 28, 34
Partition into (equal-frequency) bins:
Bin 1: 4, 8, 15
Bin 2: 21, 21, 24
Bin 3: 25, 28, 34

Smoothing by bin means:
Bin 1: 9, 9, 9
Bin 2: 22, 22, 22
Bin 3: 29, 29, 29

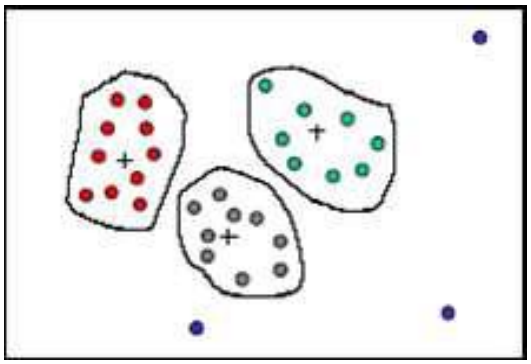
Smoothing by bin boundaries:
Bin 1: 4, 4, 15
Bin 2: 21, 21, 24
Bin 3: 25, 25, 34

1.**Binning:** Binning methods smooth a sorted data value by consulting its "neighborhood," that is, the values around it. The sorted values are distributed into a number of "buckets," or bins. Because binning methods consult the neighborhood of values, they perform local smoothing. In this example, the data for price are first sorted and then partitioned into equal-frequency bins of size 3 (i.e., each bin contains three values).

In smoothing by bin means, each value in a bin is replaced by the mean value of the bin. For example, the mean of the values 4, 8, and 15 in Bin 1 is 9. Therefore, each original value in this bin is replaced by the value 9. Similarly, smoothing by bin medians can be employed, in which each bin value is replaced by the bin median. In smoothing by bin boundaries, the minimum and maximum values in a given bin are identified as the bin boundaries. Each bin value is then replaced by the closest boundary value.
2. **Regression:** Data can be smoothed by fitting the data to a function, such as with regression. Linear regression involves finding the "best" line to fit two attributes (or variables), so that one attribute can be used to predict the other. Multiple linear regression is an extension of linear regression, where more than two attributes are involved and the data are fit to a multidimensional surface.




3. **Clustering:** Outliers may be detected by clustering, where similar values are organized into groups, or "clusters." Intuitively, values that fall outside of the set of clusters may be considered outliers



The data should also be examined regarding unique rules, consecutive rules, and null rules.

A unique rule says that each value of the given attribute must be different from all other values for that attribute.

A consecutive rule says that there can be no missing values between the lowest and highest values for the attribute, and that all values must also be unique (e.g., as in check numbers).

A null rule specifies the use of blanks, question marks, special characters, or other strings that may indicate the null condition (e.g., where a value for a given attribute is not available), and how such values should be handled

**Data Integration and Transformation**

Data mining often requires data integration -the merging of data from multiple data stores. The data may also need to be transformed into forms appropriate for mining

**Data Integration**

It is likely that your data analysis task will involve data integration, which combines data from multiple sources into a coherent data store, as in data warehousing. These sources may include multiple databases, data cubes, or flat files.

There are a number of issues to consider during data integration.

Schema integration and object matching can be tricky. How can equivalent real-world entities from multiple data sources be matched up? This is referred to as the entity identification problem.

For example, how can the data analyst or the computer be sure that customer id in one database and cust number in another refer to the same attribute?

metadata for each attribute include the name, meaning, data type, and range of values permitted for the attribute, and null rules for handling blank, zero, or null values. Such metadata can be used to help avoid errors in schema integration.

The metadata may also be used to help transform the data (e.g., where data codes for pay type in one database may be “H” and “S”, and 1 and 2 in another).

Redundancy is another important issue.
An attribute (such as annual revenue, for instance) may be redundant if it can be “derived” from another attribute or set of attributes. Inconsistencies in attribute or dimension naming can also cause redundancies in the resulting data set.

Some redundancies can be detected by correlation analysis. Given two attributes, such analysis can measure how strongly one attribute implies the other, based on the available data. For numerical attributes, we can evaluate the correlation between two attributes, A and B, by computing the correlation coefficient This is

$$r_{A,B} = \frac{\Sigma(A-\overline{A})(B-\overline{B})}{(n-1)\sigma_A\sigma_B}$$

$\overline{A}$ , $\overline{B}$ are respective mean values of A and B
$\sigma_A$, $\sigma_B$ are respective standard deviation of A and B
$n$ is the number of tubles

The result of the equation is > 0, then A and B are positively correlated, which means the value of A increases as the values of B increases. The higher value may indicate redundancy that may be removed.
The result of the equation is = 0, then A and B are independent and there is no correlation between them.
If the resulting value is < 0, then A and B are negatively correlated where the values of one attribute increase as the value of one attribute decrease which means each attribute may discourages each other.
-also called Pearson‘s product moment coefficient
For categorical (discrete) data, a correlation relationship between two attributes, A and B, can be discovered by a X2 (chi-square) test.
Suppose A has c distinct values, namely a1;a2; : : :ac. B has r distinct values, namely b1;b2; : : :br. The data tuples described by A and B can be shown as a contingency table, with the c values of A making up the columns and the r values of B making up the rows.
Let (Ai;Bj) denote the event that attribute A takes on value ai and attribute B takes on value bj, that is, where (A = ai;B = bj). Each and every possible (Ai;Bj) joint event has its own cell (or slot) in the table. The c2 value (also known as the Pearson c2 statistic) is computed as:

$$\chi^2 = \sum_{i=1}^{c}\sum_{j=1}^{r}\frac{(o_{ij}-e_{ij})^2}{e_{ij}},$$

where oi j is the observed frequency (i.e., actual count) of the joint event (Ai;Bj) and ei j is the expected frequency of (Ai;Bj), which can be computed as

$$e_{ij} = \frac{count(A = a_i) \times count(B = b_j)}{N},$$

where N is the number of data tuples, count(A=ai) is the number of tuples having value ai for A, and count(B = bj) is the number of tuples having value bj for B.

**DATA TRANSFORAMTION**

In data transformation, the data are transformed or consolidated into forms appropriate for mining.
Data transformation can involve the following:

- Smoothing, this works to remove noise from the data. Such techniques include binning, regression, and clustering.
- Aggregation, where summary or aggregation operations are applied to the data. For example, the daily sales data may be aggregated so as to compute monthly and annual total amounts. This step is typically used in constructing a data cube for analysis of the data at multiple granularities.
- Generalization of the data, where low-level or "primitive" (raw) data are replaced by higher-level concepts through the use of concept hierarchies. For example, categorical attributes, like street, can be generalized to higher-level concepts,
  like city or country. Similarly, values for numerical attributes, like age, may be mapped to higher-level concepts, like youth, middle-aged, and senior.
- Normalization, where the attribute data are scaled so as to fall within a small specified range, such as -1.0 to 1.0, or 0.0 to 1.0.
- Attribute construction (or feature construction),where new attributes are constructed and added from the given set of attributes to help the mining process.

An attribute is normalized by scaling its values so that they fall within a small specified range, such as 0.0 to 1.0.
There are many methods for data normalization.
We study three:
1. min-max normalization
2. z-score normalization
3. normalization by decimal scaling.

**Min-max normalization** performs a linear transformation on the original data. Suppose that minA and maxA are the minimum and maximum values of an attribute, Min-max normalization maps a value, v, of A to v1 in the range [new minA;new maxA] by computing

$$v' = \frac{v - min_A}{max_A - min_A}(new_max_A - new_min_A) + new_min_A.$$

Suppose that the minimum and maximum values for the attribute income are $12,000 and $98,000, respectively. We would like to map income to the range [0.0,1.0].
By min-max normalization, a value of $73,600 for income is transformed to

73,600-12,000/
98,000-12,000 (1.0-0)+0 = 0.716.

In **z-score normalization** (or zero-mean normalization), the values for an attribute, A, are normalized based on the mean and standard deviation of A. A value, v, of A is normalized to v1 by computing

$$v' = \frac{v - \bar{A}}{\sigma_A}, \quad (2.12)$$

where $\bar{A}$ and $\sigma_A$ are the mean and standard deviation, respectively, of attribute $A$. This method of normalization is useful when the actual minimum and maximum of attribute $A$ are unknown, or when there are outliers that dominate the min-max normalization.

z-score normalization Suppose that the mean and standard deviation of the values for the attribute income are $54,000 and $16,000, respectively. With z-score normalization, a value of $73,600 for income is transformed to 73,600-54,000/16,000 = 1.225.

**Normalization by decimal scaling** normalizes by moving the decimal point of values of attribute A. The number of decimal points moved depends on the maximum absolute value of A. A value, v, of A is normalized to v1 by computing Decimal scaling.

$$v' = \frac{v}{10^j},$$

where $j$ is the smallest integer such that $Max(|v'|) < 1$.

Suppose that the recorded values of A range from -986 to 917. The maximum absolute value of A is 986. To normalize by decimal scaling, we therefore divide each value by 1,000 (i.e., j = 3) so that -986 normalizes to -0.986 and 917 normalizes to 0.917.

**Data Reduction**

Imagine that you have selected data from the AllElectronics data warehouse for analysis. The data set will likely be huge! Complex data analysis and mining on huge amounts of data can take a long time, making such analysis impractical or infeasible.

Data reduction techniques can be applied to obtain a reduced representation of the data set that is much smaller in volume, yet closely maintains the integrity of the original data.

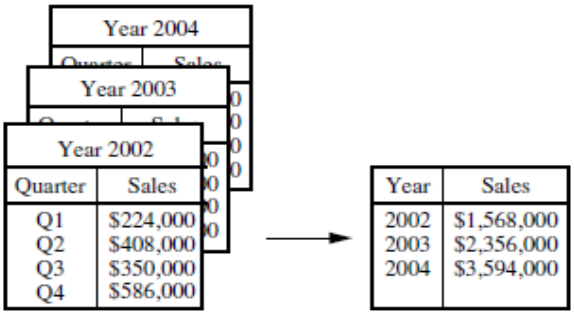
Strategies for data reduction include the following:

1. **Data cube aggregation**, where aggregation operations are applied to the data in the construction of a data cube.
2. **Attribute subset selection**, where irrelevant, weakly relevant, or redundant attributes or dimensions may be detected and removed.
3. **Dimensionality reduction**, where encoding mechanisms are used to reduce the data set size.
4. **Numerosity reduction**, here the data are replaced or estimated by alternative, smaller data representations such as parametric models (which need store only the model parameters instead of the actual data) or nonparametric methods such as clustering, sampling, and the use of histograms.
5. **Discretization and concept hierarchy generation**, where raw data values for attributes are replaced by ranges or higher conceptual levels. Data discretization is a form of numerosity reduction that is very useful for the automatic generation of concept hierarchies.

**Data Cube Aggregation**

Imagine that you have collected the data for your analysis. These data consist of the AllElectronics sales

per quarter, for the years 2002 to 2004. You are, however, interested in the annual sales (total per year), rather than the total per quarter. Thus the data can be aggregated so that the resulting data summarize the total sales per year instead of per quarter.



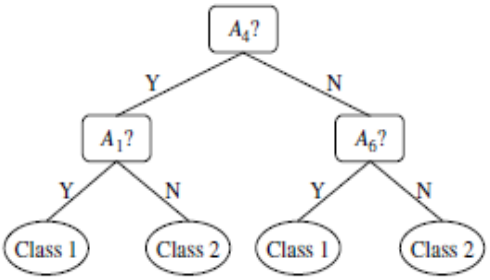

**Attribute Subset Selection**

Attribute subset selection reduces the data set size by removing irrelevant or redundant attributes (or dimensions). The goal of attribute subset selection is to find a minimum set of attributes such that the resulting probability distribution of the data classes is as close as possible to the original distribution obtained using all attributes.

Mining on a reduced set of attributes has an additional benefit. It reduces the number of attributes appearing in the discovered patterns, helping to make the patterns easier to understand.

"How can we find a 'good' subset of the original attributes?" For n attributes, there are 2 power n possible subsets.

Basic heuristic methods of attribute subset selection include the following techniques

| Forward selection | Backward elimination | Decision tree induction |
|---|---|---|
| Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$<br><br>Initial reduced set: $\{\}$<br>=> $\{A_1\}$<br>=> $\{A_1, A_4\}$<br>=> Reduced attribute set: $\{A_1, A_4, A_6\}$ | Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$<br><br>=> $\{A_1, A_3, A_4, A_5, A_6\}$<br>=> $\{A_1, A_4, A_5, A_6\}$<br>=> Reduced attribute set: $\{A_1, A_4, A_6\}$ | Initial attribute set: $\{A_1, A_2, A_3, A_4, A_5, A_6\}$<br>  <br>=> Reduced attribute set: $\{A_1, A_4, A_6\}$ |



1. Stepwise forward selection: The procedure starts with an empty set of attributes as the reduced set. The best of the original attributes is determined and added to the reduced set. At each subsequent iteration or step, the best of the remaining original attributes is added to the set.
2. Stepwise backward elimination: The procedure starts with the full set of attributes. At each step, it removes the worst attribute remaining in the set.
3. Combination of forward selection and backward elimination: The stepwise forward selection and backward elimination methods can be combined so that, at each step, the procedure selects the best attribute and removes the worst from among the remaining attributes.

4. Decision tree induction constructs a flowchart like structure where each internal (non leaf) node denotes a test on an attribute, each branch corresponds to an outcome of the test, and each external (leaf) node denotes a class prediction. At each node, the algorithm chooses the "best" attribute to partition the data into individual classes. When decision tree induction is used for attribute subset selection, a tree is constructed from the given data. All attributes that do not appear in the tree are assumed to be irrelevant. The set of attributes appearing in the tree form the reduced subset of attributes.

**Dimensionality Reduction**

In dimensionality reduction, data encoding or transformations are applied so as to obtain a reduced or "compressed" representation of the original data.

loss less:

If the original data can be reconstructed from the compressed data without any loss of information, the data reduction is called lossless.

lossy:

If, instead, we can reconstruct only an approximation of the original data, then the data reduction is called lossy.

effective methods of lossy dimensionality reduction are

1. wavelet transforms and
2. principal components analysis.

**Wavelet Transforms**

The discrete wavelet transform(DWT) is a linear signal processing technique that, when applied to a data vector X, transforms it to a numerically different vector, X-1, of wavelet coefficients. The two vectors are of the same length.

When applying this technique to data reduction, we consider each tuple as an n-dimensional data vector, that is, X = (x1,x2, … ,xn), depicting n measurements made on the tuple from n database attributes.

"How can this technique be useful for data reduction if the wavelet transformed data are of the same length as the original data?" The usefulness lies in the fact that the wavelet transformed data can be truncated.

A compressed approximation of the data can be retained by storing only a small fraction of the strongest of the wavelet coefficients.

For example, all wavelet coefficients larger than some user-specified threshold can be retained. All other coefficients are set to 0.

**Principal Components Analysis**

Suppose that the data to be reduced consist of tuples or data vectors described by n attributes or dimensions.

Principal components analysis, or PCA (also called the Karhunen-Loeve, or K-L, method), searches for k n-dimensional orthogonal vectors that can best be used to represent the data, where k <= n. The original data are thus projected onto a much smaller space, resulting in dimensionality reduction.

**Numerosity Reduction**

"Can we reduce the data volume by choosing alternative, 'smaller' forms of data representation?" .Techniques of numerosity reduction may be

1. Parametric
2. Non parametric.

For parametric methods, a model is used to estimate the data, so that typically only the data parameters need to be stored, instead of the actual data.

Log-linear models, which estimate discrete multidimensional probability distributions, are an example.

Nonparametric methods for storing reduced representations of the data include histograms, clustering, and

sampling.

**Regression and Log-Linear Models**

Regression and log-linear models can be used to approximate the given data. In (simple) linear regression, the data are modeled to fit a straight line. For example, a random variable, y (called a response variable), can be modeled as a linear function of another random variable, x (called a predictor variable), with the equation

$$y = w\,x + b$$

Multiple linear regression is an extension of (simple) linear regression, which allows a response variable, y, to be modeled as a linear function of two or more predictor variables.

Log-linear models approximate discrete multidimensional probability distributions. Given a set of tuples in n dimensions (e.g., described by n attributes), we can consider each tuple as a point in an n-dimensional space. Log-linear models can be used to estimate the probability of each point in a multidimensional space for a set of discretized attributes, based on a smaller subset of dimensional combinations.

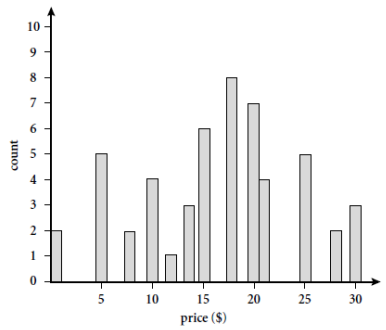
**Histograms**

Histograms use binning to approximate data distributions and are a popular form of data reduction.

A histogram for an attribute, A, partitions the data distribution of A into disjoint subsets, or buckets. If each bucket represents only a single attribute-value/frequency pair, the buckets are called singleton buckets. Often, buckets instead represent continuous ranges for the given attribute.

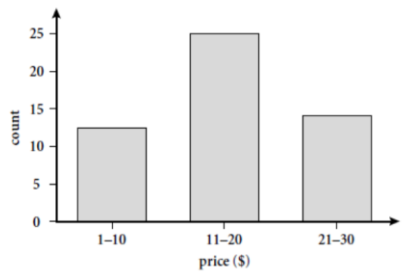
Ex

Histograms. The following data are a list of prices of commonly sold items at AllElectronics (rounded to the nearest dollar). The numbers have been sorted: 1, 1, 5, 5, 5, 5, 5, 8, 8, 10, 10, 10, 10, 12, 14, 14, 14, 15, 15, 15, 15, 15, 15, 18, 18, 18, 18, 18, 18, 18, 18, 20, 20, 20, 20, 20, 20, 20, 21, 21, 21, 21, 25, 25, 25, 25, 25, 28, 28, 30, 30, 30.




“How are the buckets determined and the attribute values partitioned?” There are several partitioning rules, including the following:

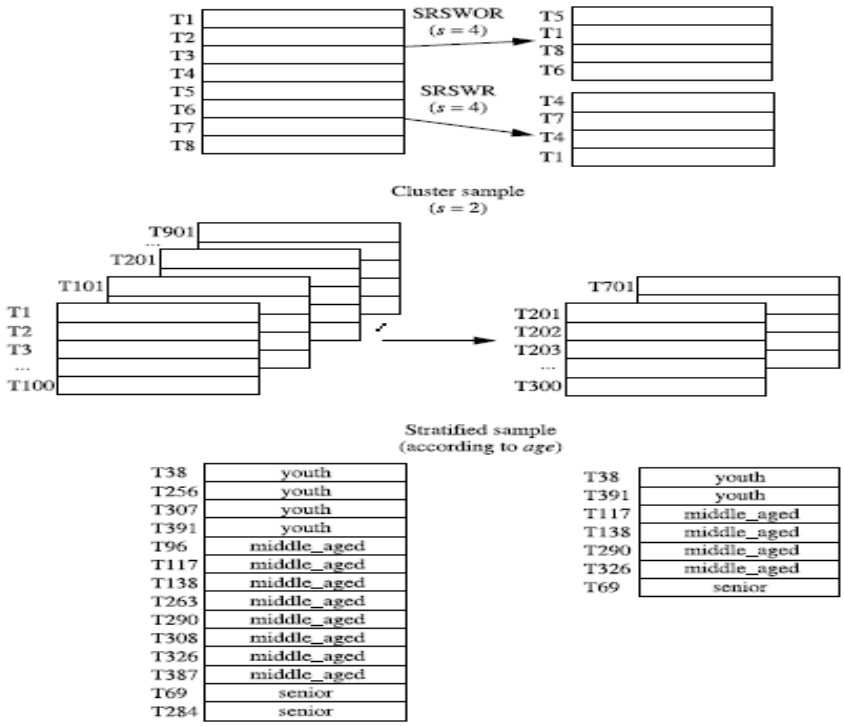
**Equal-width:** In an equal-width histogram, the width of each bucket range is uniform



**Equal-frequency (or equidepth):** In an equal-frequency histogram, the buckets are created so that, roughly, the frequency of each bucket is constant (that is, each bucket contains roughly the same number of contiguous data samples).
**V-Optimal:** If we consider all of the possible histograms for a given number of buckets, the V-Optimal histogram is the one with the least variance. Histogram variance is a weighted sum of the original values that each bucket represents, where bucket weight is equal to the number of values in the bucket.
**MaxDiff:** In a MaxDiff histogram, we consider the difference between each pair of adjacent values. A bucket boundary is established between each pair for pairs having the b-1 largest differences, where b is the user-specified number of buckets.

**Clustering**
Clustering techniques consider data tuples as objects. They partition the objects into groups or clusters, so that objects within a cluster are "similar" to one another and "dissimilar" to objects in other clusters. Similarity is commonly defined in terms of how "close" the objects are in space, based on a distance function.
The "quality" of a cluster may be represented by its diameter, the maximum distance between any two objects in the cluster. Centroid distance is an alternative measure of cluster quality and is defined as the average distance of each cluster object from the cluster centroid.
In data reduction, the cluster representations of the data are used to replace the actual data.

**Sampling**
Sampling can be used as a data reduction technique because it allows a large data set to be represented by a much smaller random sample (or subset) of the data. Suppose that a large data set, D, contains N tuples. Let's look at the most common ways that we could sample D for data reduction,



**Simple random sample without replacement (SRSWOR) of size s:** This is created by drawing s of the N tuples from D (s < N), where the probability of drawing any tuple in D is 1/N, that is, all tuples are equally likely to be sampled.

**Simple random sample with replacement (SRSWR) of size s:** This is similar to SRSWOR, except that each time a tuple is drawn from D, it is recorded and then replaced. That is, after a tuple is drawn, it is placed back in D so that it may be drawn again.

**Cluster sample:** If the tuples in D are grouped into M mutually disjoint "clusters," then an SRS of s clusters can be obtained, where s < M. For example, tuples in a database are usually retrieved a page at a time, so that each page can be considered a cluster. A reduced data representation can be obtained by applying, say, SRSWOR to the pages, resulting in a cluster sample of the tuples. Other clustering criteria conveying rich semantics can also be explored. For example, in a spatial database, we may choose to define clusters geographically based on how closely different areas are located.

**Stratified sample:** If D is divided into mutually disjoint parts called strata, a stratified sample of D is generated by obtaining an SRS at each stratum. This helps ensure a representative sample, especially when the data are skewed. For example, a stratified sample may be obtained from customer data, where a stratum is created for each customer age group. In this way, the age group having the smallest number of customers will be sure to be represented.

**Data Discretization and Concept Hierarchy Generation**

Data discretization techniques can be used to reduce the number of values for a given continuous attribute by dividing the range of the attribute into intervals. Interval labels can then be used to replace actual data values.

Discretization techniques can be categorized based on how the discretization is performed

1. Top-down
2. Bottom-up

If the discretization process uses class information, then we say it is supervised discretization. Otherwise, it is unsupervised.

If the process starts by first finding one or a few points (called split points or cut points) to split the entire attribute range, and then repeats this recursively on the resulting intervals, it is called top-down discretization or splitting.

The bottom-up discretization or merging, which starts by considering all of the continuous values as potential split-points, removes some by merging neighborhood values to form intervals, and then recursively applies this process to the resulting intervals.

A concept hierarchy for a given numerical attribute defines a discretization of the attribute. Concept hierarchies can be used to reduce the data by collecting and replacing low-level concepts (such as numerical values for the attribute age) with higher-level concepts (such as youth, middle-aged, or senior).

**Discretization and Concept Hierarchy Generation for Numerical Data**

Concept hierarchies for numerical attributes can be constructed automatically based on data discretization.

**Binning**

Binning is a top-down splitting technique based on a specified number of bins. For example, attribute values can be discretized by applying equal-width or equal-frequency binning, and then replacing each bin value by the bin mean or median, as in smoothing by bin means or smoothing by bin medians, respectively.

**Histogram Analysis**
Histograms partition the values for an attribute, A, into disjoint ranges called buckets.

**Entropy-Based Discretization**
Entropy is one of the most commonly used discretization measures. Entropy-based discretization is a supervised, top-down splitting technique. It explores class distribution information in its calculation and determination of split-points.
To discretize a numerical attribute, A, the method selects the value of A that has the minimum entropy as a split-point, and recursively partitions the resulting intervals to arrive at a hierarchical discretization. Such discretization forms a concept hierarchy for A.
Let D consist of data tuples defined by a set of attributes and a class-label attribute.
The class-label attribute provides the class information per tuple. The basic method for entropy-based discretization of an attribute A within the set is as follows:

1. Each value of A can be considered as a potential interval boundary or split-point
(denoted split point) to partition the range of A. That is, a split-point for A can partition the tuples in D into two subsets satisfying the conditions A <= split point and A > split point, respectively, thereby creating a binary discretization.
2. Suppose we want to classify the tuples in D by partitioning on attribute A and some split-point. Ideally, we would like this partitioning to result in an exact classification of the tuples. For example, if we had two classes, we would hope that all of the tuples of, say, class C1 will fall into one partition and all of the tuples of class C2 will fall into the other partition. However, this is unlikely. For example, the first partition may contain many tuples of C1, but also some of C2. How much more information would we still need for a perfect classification, after this partitioning? This amount is called the expected information requirement for classifying a tuple in D based on partitioning by A. It is given by

$$Info_A(D) = \frac{|D_1|}{|D|} Entropy(D_1) + \frac{|D_2|}{|D|} Entropy(D_2),$$

Where D1 and D2 correspond to the tuples in D satisfying the conditions A <= split point and A > split point, respectively; |D| is the number of tuples in D, and so on. The entropy function for a given set is calculated based on the class distribution of the tuples in the set. For example, given m classes, C1, C2… Cm, the entropy of D1 is

$$Entropy(D_1) = -\sum_{i=1}^{m} p_i \log_2(p_i),$$

Where pi is the probability of class Ci in D1, determined by dividing the number of tuples of class Ci in D1 by |D1|, the total number of tuples in D1. Therefore, when selecting a split-point for attribute A, we want to pick the attribute value that gives the minimum expected information requirement (i.e., min(InfoA(D))).

3. The process of determining a split-point is recursively applied to each partition obtained, until some stopping criterion is met, such as when the minimum information requirement on all candidate split-points is less than a small threshold, e, or when the number of intervals is greater than a threshold, max interval.

**Interval Merging by $X^2$ Analysis**
*ChiMerge is a* $X^2$ based discretization method. The discretization methods that we have studied up to this point have all employed a top-down, splitting strategy. This contrasts with ChiMerge, which employs a

bottom-up approach by finding the best neighboring intervals and then merging these to form larger intervals, recursively. The method is supervised in that it uses class information.

**Cluster Analysis**

Cluster analysis is a popular data discretization method. A clustering algorithm can be applied to discretize a numerical attribute, A, by partitioning the values of A into clusters or groups. Clustering takes the distribution of A into consideration, as well as the closeness of data points, and therefore is able to produce high-quality discretization results.

**Concept Hierarchy Generation for Categorical Data**

Categorical data are discrete data. Categorical attributes have a finite (but possibly large) number of distinct values, with no ordering among the values. Examples include geographic location, job category, and item type.

There are several methods for the generation of concept hierarchies for categorical data.

1. **Specification of a partial ordering of attributes explicitly at the schema level by users or experts**:

Concept hierarchies for categorical attributes or dimensions typically involve a group of attributes. A user or expert can easily define a concept hierarchy by specifying a partial or total ordering of the attributes at the schema level. For example,

a relational database or a dimension location of a data warehouse may contain the following group of attributes: street, city, province or state, and country. A hierarchy can be defined by specifying the total ordering among these attributes at the schema level, such as street < city < province or state < country.

2. **Specification of a portion of a hierarchy by explicit data grouping:**

after specifying that province and country form a hierarchy at the schema level, a user could define some intermediate levels manually, such as "{Alberta, Saskatchewan, Manitobag C prairies Canada}" and "{British Columbia, prairies Canadag C Western Canada}".

3. **Specification of a set of attributes, but not of their partial ordering:**

A user may specify a set of attributes forming a concept hierarchy, but omit to explicitly state their partial ordering. The system can then try to automatically generate the attribute ordering so as to construct a meaningful concept hierarchy.

4. **Specification of only a partial set of attributes:**

Sometimes a user can be sloppy when defining a hierarchy, or have only a vague idea about what should be included in a hierarchy. Consequently, the user may have included only a small subset of the relevant attributes in the hierarchy specification. For example, instead of including all of the hierarchically relevant attributes for location, the user may have specified only street and city.

To handle such partially specified hierarchies, it is important to embed data semantics in the database schema so that attributes with tight semantic connections can be pinned together. In this way, the specification of one attribute may trigger a whole group of semantically tightly linked attributes to be "dragged in" to form a complete hierarchy.

**Unit 2**

Warehousing: Data Warehouse and OLAP Technology for Data Mining Data Warehouse, Multidimensional Data Model, Data Warehouse Architecture

Mining Association Rules In Large Databases: Basic Concepts and a Road Map, Efficient and Scalable Frequent Item set Mining Methods, Mining Various Kinds of Association Rules, From Association Mining to Correlation Analysis.

## INTRODUCTION TO DATA WAREHOUSE

A data warehouse is a collection of data marts representing historical data from different operations in the company. This data is stored in a structure optimized for querying and data analysis as a data warehouse. A data warehouse can also be viewed as a database for historical data from different functions within a company.

The term Data Warehouse was coined by Bill Inmon in 1990, which he defined in the following way: "A warehouse is a subject-oriented, integrated, time-variant and non- volatile collection of data in support of management's decision making process".

He defined the terms in the sentence as follows:

**Subject Oriented:** A data warehouse is organized around major subjects, such as customer, supplier, product, and sales. Rather than concentrating on the day-to-day operations and transaction processing of an organization, a data warehouse focuses on the modeling and analysis of data for decision makers.

**Integrated:** A data warehouse is usually constructed by integrating multiple heterogeneous sources, such as relational databases, flat files, and on-line transaction records. Data cleaning and data integration techniques are applied to ensure consistency in naming conventions, encoding structures, attribute measures, and so on.

**Time-variant:** Data are stored to provide information from a historical perspective (e.g., the past 5–10 years).

**Non-volatile:** A data warehouse is always a physically separate store of data transformed from the application data found in the operational environment. Due to this separation, a data warehouse does not require transaction processing, recovery, and concurrency control mechanisms. It usually requires only two operations in data accessing: initial loading of data and access of data.

"*How are organizations using the information from data warehouses?*"

Many organizations use this information to support business decision-making activities, including

(1) increasing customer focus, which includes the analysis of customer buying patterns (such as buying preference, buying time, budget cycles, and appetites for spending);

(2) repositioning products and managing product portfolios by comparing the performance of sales by quarter, by year, and by geographic regions in order to fine tune production strategies;

(3) analyzing operations and looking for sources of profit;

(4) managing the customer relationships, making environmental corrections, and managing the cost

of corporate assets.

**Differences between Operational Database Systems and DataWarehouses**

The major task of on-line operational database systems is to perform on-line transaction and query processing. These systems are called on-line transaction processing (OLTP) systems. They cover most of the day-to-day operations of an organization, such as purchasing, inventory, manufacturing, banking, payroll, registration, and accounting.

Data warehouse systems, on the other hand, serve users or knowledge workers in the role of data analysis and decision making. Such systems can organize and present data in various formats in order to accommodate the diverse needs of the different users. These systems are known as on-line analytical processing (OLAP) systems.

**The major distinguishing features between OLTP and OLAP are summarized as follows:**

**Users and system orientation**: An OLTP system is customer-oriented and is used for transaction and query processing by clerks, clients, and information technology professionals. An OLAP system is market-oriented and is used for data analysis by knowledge workers, including managers, executives, and analysts.

**Data contents**: An OLTP system manages current data that, typically, are too detailed to be easily used for decision making. An OLAP system manages large amounts of historical data, provides facilities for summarization and aggregation, and stores and manages information at different levels of granularity. These features make the data easier to use in informed decision making.

**Database design**: An OLTP system usually adopts an entity-relationship (ER) data model and an application-oriented database design. An OLAP system typically adopts either a star or snowflake model and a subject oriented database design.

**View:** An OLTP system focuses mainly on the current data within an enterprise or department, without referring to historical data or data in different organizations. In contrast, an OLAP system often spans multiple versions of a database schema, due to the evolutionary process of an organization. OLAP systems also deal with information that originates from different organizations, integrating information from many data stores. Because of their huge volume, OLAP data are stored on multiple storage media.

**Access patterns**: The access patterns of an OLTP system consist mainly of short, atomic transactions. Such a system requires concurrency control and recovery mechanisms. However, accesses to OLAP systems are mostly read-only operations (because most data warehouses store historical rather than up-to-date information), although many could be complex queries.

| Feature | OLTP | OLAP |
|---|---|---|
| Characteristic | operational processing | informational processing |
| Orientation | transaction | analysis |
| User | clerk, DBA, database professional | knowledge worker (e.g., manager, executive, analyst) |
| Function | day-to-day operations | long-term informational requirements, decision support |
| DB design | ER based, application-oriented | star/snowflake, subject-oriented |
| Data | current; guaranteed up-to-date | historical; accuracy maintained over time |
| Summarization | primitive, highly detailed | summarized, consolidated |
| View | detailed, flat relational | summarized, multidimensional |
| Unit of work | short, simple transaction | complex query |
| Access | read/write | mostly read |
| Focus | data in | information out |
| Operations | index/hash on primary key | lots of scans |
| Number of records accessed | tens | millions |
| Number of users | thousands | hundreds |
| DB size | 100 MB to GB | 100 GB to TB |
| Priority | high performance, high availability | high flexibility, end-user autonomy |
| Metric | transaction throughput | query throughput, response time |

## A MULTIDIMENSIONAL DATA MODEL

Data warehouses and OLAP tools are based on a multidimensional data model. This model views data in the form of a data cube.

### From Tables and Spreadsheets to Data Cubes

"What is a data cube?" A data cube allows data to be modeled and viewed in multiple dimensions. It is defined by dimensions and facts.

In general terms, dimensions are the perspectives or entities with respect to which an organization wants to keep records.

For example, AllElectronics may create a sales data warehouse in order to keep records of the store's sales with respect to the dimensions time, item, branch, and location. These dimensions allow the store to keep track of things like monthly sales of items and the branches and locations at which the items were sold.

Each dimension may have a table associated with it, called a dimension table, which further describes the dimension. For example, a dimension table for item may contain the attributes item name, brand, and type.

A multidimensional data model is typically organized around a central theme, like sales, for instance. This theme is represented by a **fact table**. Facts are numerical measures. Think of them as the quantities by

which we want to analyze relationships between dimensions.

Examples of facts for a sales data warehouse include dollars sold (sales amount in dollars), units sold (number of units sold), and amount budgeted. The fact table contains the names of the facts, or measures, as well as keys to each of the related dimension tables.

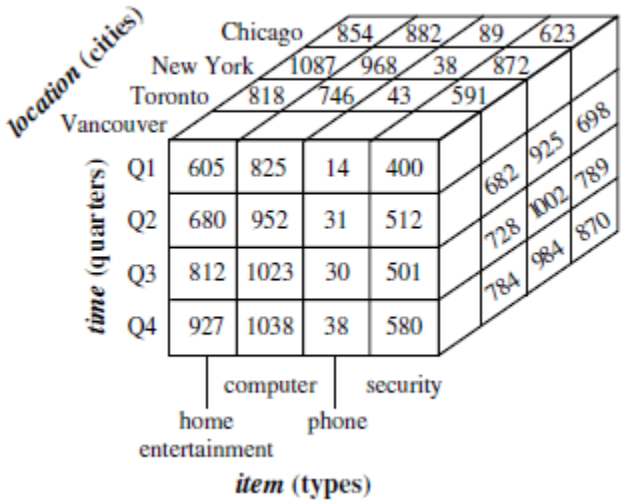
To gain a better understanding of data cubes and the multidimensional data model, let's start by looking at a simple 2-D data cube that is, in fact, a table or spreadsheet for sales data from AllElectronics. In particular, we will look at the AllElectronics sales data for items sold per quarter in the city of Vancouver.

| | **location** = "Vancouver" | | | |
|---|---|---|---|---|
| | **item** (type) | | | |
| **time** (quarter) | home entertainment | computer | phone | security |
| Q1 | 605 | 825 | 14 | 400 |
| Q2 | 680 | 952 | 31 | 512 |
| Q3 | 812 | 1023 | 30 | 501 |
| Q4 | 927 | 1038 | 38 | 580 |

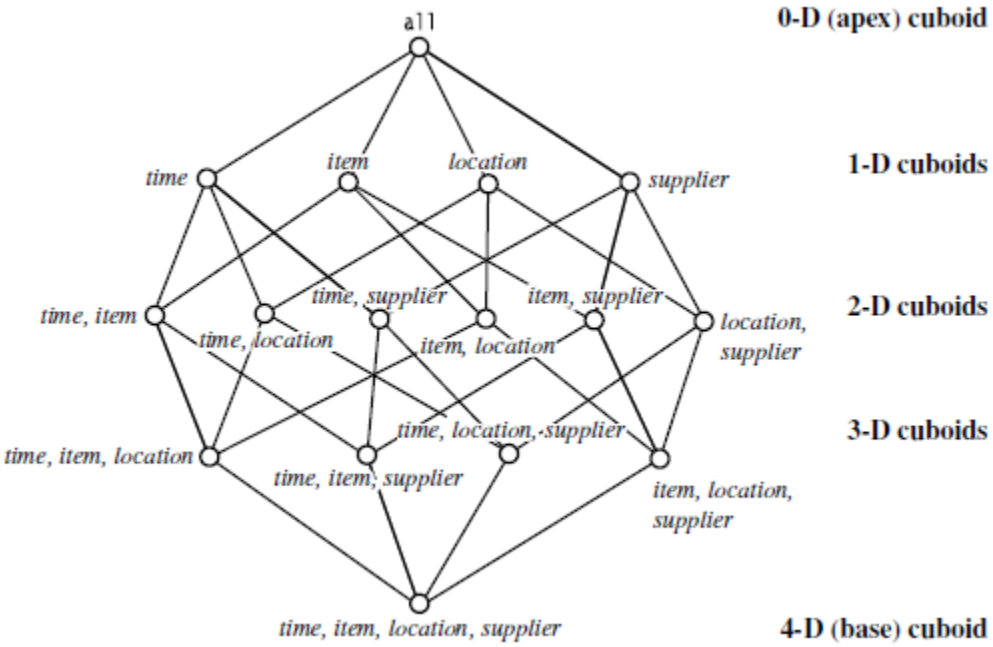
Now, suppose that we would like to view the sales data with a third dimension. For instance, suppose we would like to view the data according to time and item, as well as location for the cities Chicago, New York, Toronto, and Vancouver. The 3-D data of Table are represented as a series of 2-D tables.

| | **location** = "Chicago" | | | | **location** = "New York" | | | | **location** = "Toronto" | | | | **location** = "Vancouver" | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| | **item** | | | | **item** | | | | **item** | | | | **item** | | | |
| | home | | | | home | | | | home | | | | home | | | |
| **time** | ent. | comp. | phone | sec. | ent. | comp. | phone | sec. | ent. | comp. | phone | sec. | ent. | comp. | phone | sec. |
| Q1 | 854 | 882 | 89 | 623 | 1087 | 968 | 38 | 872 | 818 | 746 | 43 | 591 | 605 | 825 | 14 | 400 |
| Q2 | 943 | 890 | 64 | 698 | 1130 | 1024 | 41 | 925 | 894 | 769 | 52 | 682 | 680 | 952 | 31 | 512 |
| Q3 | 1032 | 924 | 59 | 789 | 1034 | 1048 | 45 | 1002 | 940 | 795 | 58 | 728 | 812 | 1023 | 30 | 501 |
| Q4 | 1129 | 992 | 63 | 870 | 1142 | 1091 | 54 | 984 | 978 | 864 | 59 | 784 | 927 | 1038 | 38 | 580 |

Conceptually, we may also represent the same data in the form of a 3-D data cube, as in




Suppose that we would now like to view our sales data with an additional fourth dimension, such as Supplier. Viewing things in 4-D becomes tricky. However, we can think of a 4-D cube as being a series of 3-D cubes and by a cuboid




**Multidimensional Data Model**

The entity-relationship data model is commonly used in the design of relational databases, where a database schema consists of a set of entities and the relationships between them. Such a data model is appropriate for on-line transaction processing.

A data warehouse, however, requires a concise, subject-oriented schema that facilitates on-line data

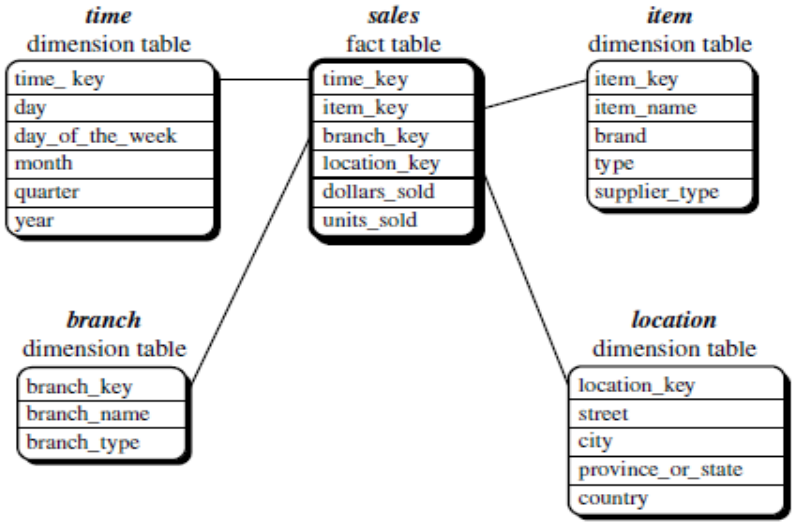
analysis.
The most popular data model for data warehouses is a multidimensional model. This model can exist in the form of a star schema, a snowflake schema, or a fact constellation schema. Let's have a look at each of these schema types.

**STAR SCHEMA**
The star schema is a modeling paradigm in which the data warehouse contains

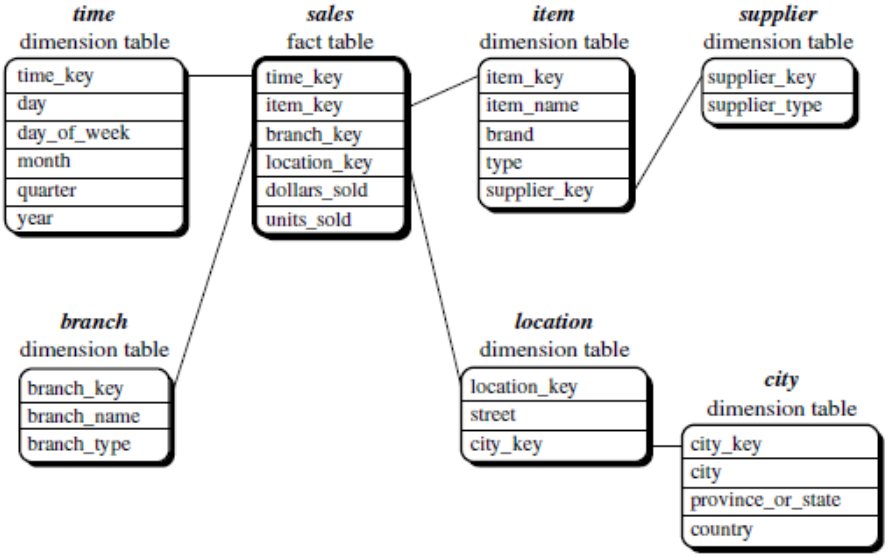
(1) a large central table (fact table)
(2) a set of smaller attendant tables (dimension tables), one for each dimension.

The schema graph resembles a starburst, with the dimension tables displayed in a radial pattern around the central fact table.

Sales are considered along four dimensions, namely, time, item, branch, and location. The schema contains a central fact table for sales that contains keys to each of the four dimensions, along with two measures: dollars sold and units sold.
To minimize the size of the fact table, dimension identifiers (such as time key and item key) are system-generated identifiers.

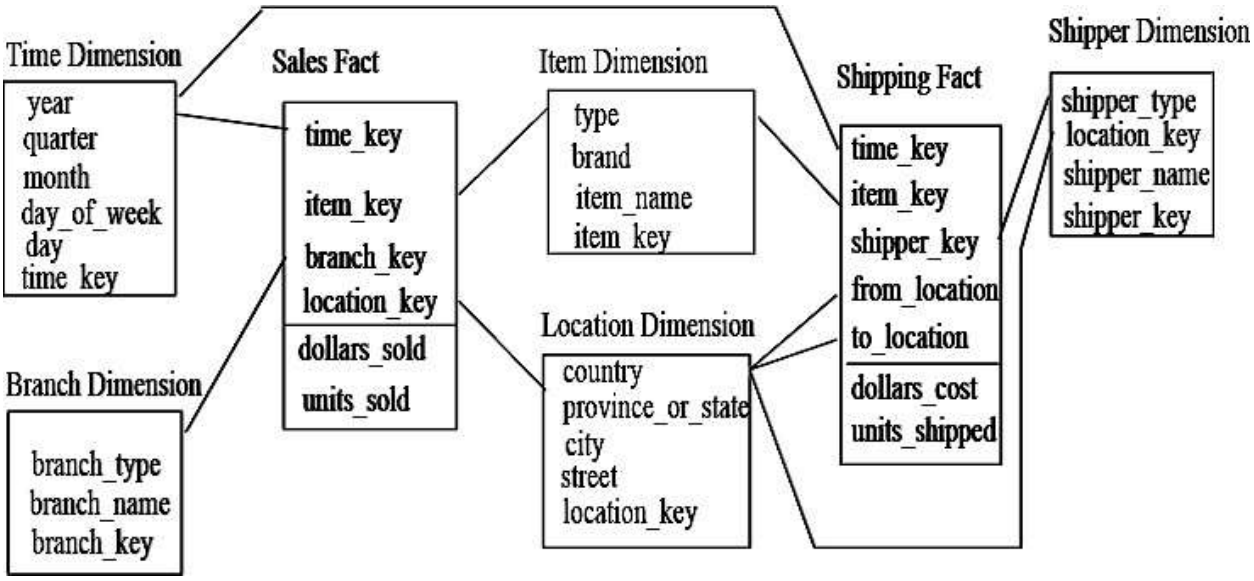



**SNOW FLAKE SCHEMA**
The snowflake schema is a variant of the star schema model, where some dimension tables are normalized, thereby further splitting the data into additional tables.
The resulting schema graph forms a shape similar to a snowflake. The major difference between the snowflake and star schema models is that the dimension tables of the snowflake model may be kept in normalized form.
Such a table is easy to maintain and also saves storage space because a large dimension table can be extremely large when the dimensional structure is included as columns.




**FACT CONSTELLATION SCHEMA**

Sophisticated applications may require multiple fact tables to share dimension tables. This kind of schema can be viewed as a collection of stars, and hence is called a galaxy schema or a fact constellation.




**Examples for Defining Star, Snowflake, and Fact Constellation Schemas**

A Data Mining Query Language can be used to specify data mining tasks.

Data warehouses and data marts can be defined using two language primitives, one for cube definition and one for dimension definition.

DMQL: Language Primitives

Cube Definition (FactTable)
• define cube<cube_name>[<dimension_list>]: <measure_list>
Dimension Definition (DimensionTable)
• define dimension <dimension_name> as(<attribute_or_subdimension_list>)

**Defining a Star Schema in DMQL**
define cube sales_star [time, item, branch, location]:
dollars_sold = sum(sales_in_dollars), avg_sales = avg(sales_in_dollars), units_sold = count(*)
define dimension time as (time_key, day, day_of_week, month, quarter, year)
define dimension item as (item_key, item_name, brand, type, supplier_type)
define dimension branch as (branch_key, branch_name, branch_type)
define dimension location as (location key, street, city, province or state,country)

**Defining a Snowflake Schema in DMQL**
define cube sales_snowflake [time, item, branch, location]:
dollars_sold = sum(sales_in_dollars), avg_sales = avg(sales_in_dollars), units_sold = count(*)
define dimension time as (time_key, day, day_of_week, month, quarter, year)
define dimension item as (item_key, item_name, brand, type, supplier(supplier_key, supplier_type))
define dimension branch as (branch_key, branch_name, branch_type)
define dimension location as (location_key, street, city(city_key, province_or_state, country))

**Defining a Fact Constellation in DMQL**
define cube sales [time, item, branch, location]:
dollars_sold = sum(sales_in_dollars), avg_sales = avg(sales_in_dollars), units_sold = count(*)
define dimension time as (time_key, day, day_of_week, month, quarter, year)
define dimension item as (item_key, item_name, brand, type, supplier_type)
define dimension branch as (branch_key, branch_name, branch_type)
define dimension location as (location_key, street, city, province_or_state, country)

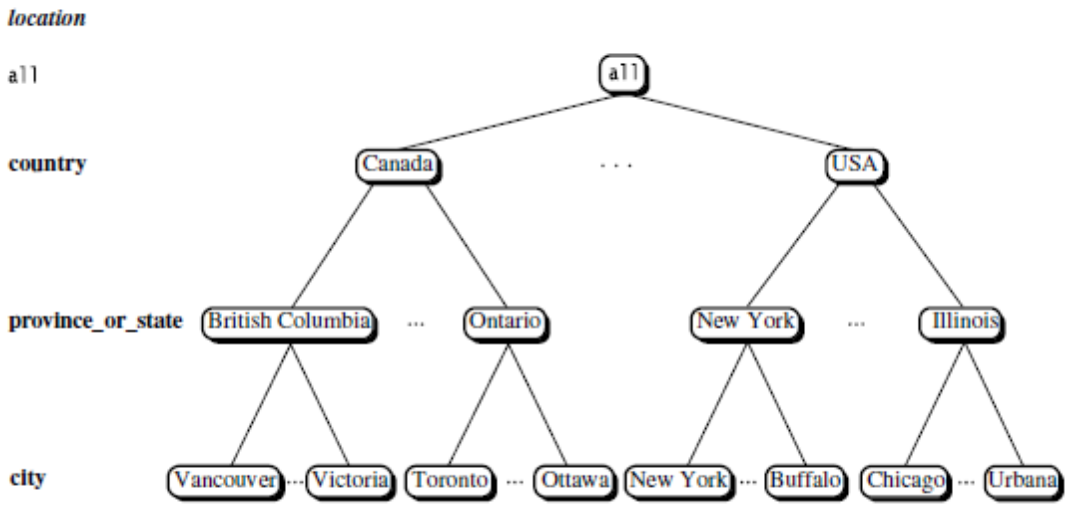
define cube shipping [time, item, shipper, from_location, to_location]:
dollar_cost = sum(cost_in_dollars), unit_shipped =count(*)
define dimension time as time in cubesales
define dimension item as item in cubesales
define dimension shipper as (shipper_key, shipper_name, location as location in cube sales, shipper_type)
define dimension from_location as location in cube sales
define dimension to_location as location in cube sales

**MEASURES**
Measures: Three Categories
Measure: a function evaluated on aggregated data corresponding to given dimension-value pairs.
Measures can be:
**distributive**: if the measure can be calculated in a distributive manner.
E.g., count(), sum(), min(),max().
**algebraic**: if it can be computed from arguments obtained by applying distributive aggregate functions.
E.g., avg() = sum()/count(), min_N(), standard_deviation().
**holistic**: if it is not algebraic.
E.g., median(), mode(),rank().

**CONCEPT HIERARCHY**

A Concept hierarchy defines a sequence of mappings from a set of low level Concepts to higher level, more general Concepts. Concept hierarchies allow data to be handled at varying levels of abstraction. Many concept hierarchies are implicit within the database schema.

For example, suppose that the dimension location is described by the attributes number, street, city, province or state, zip code, and country. These attributes are related by a total order, forming a concept hierarchy such as "street < city < province or state < country".




**OLAP operations on multidimensional data.**

"How are concept hierarchies useful in OLAP?" In the multidimensional model, data are organized into multiple dimensions, and each dimension contains multiple levels of abstraction defined by concept hierarchies. This organization provides users with the flexibility to view data from different perspectives.

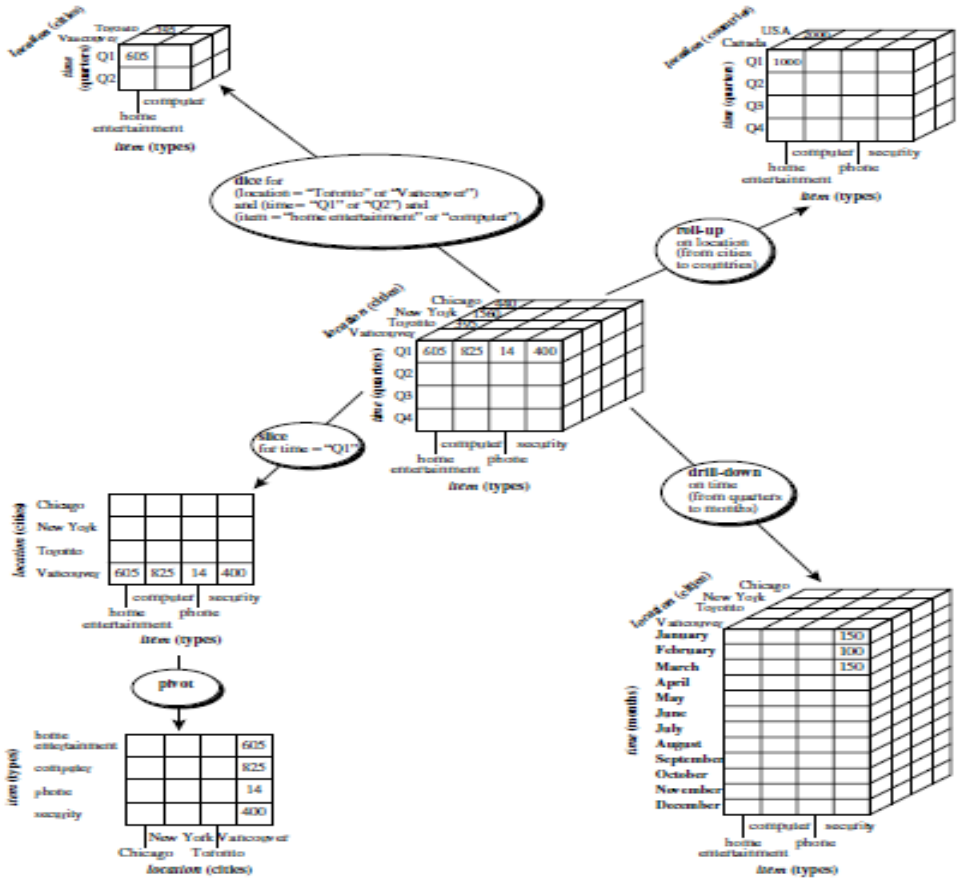
**Roll-up**: The roll-up operation performs aggregation on a data cube, either by climbing-up a concept hierarchy for a dimension or by dimension reduction. Figure shows the result of a roll-up operation performed on the central cube by climbing up the concept hierarchy for location. This hierarchy was defined as the total order street < city < province or state<country.

**Drill-down:**

Drill-down is the reverse of roll-up. It navigates from less detailed data to more detailed data. Drill-down can be realized by either stepping-down a concept hierarchy for a dimension or introducing additional dimensions. Figure shows the result of a drill-down operation performed on the central cube by stepping down a concept hierarchy for time defined as day < month < quarter <year. Drill-down occurs by descending the time hierarchy from the level of quarter to the more detailed level of month.

**Slice and dice:** The slice operation performs a selection on one dimension of the given cube, resulting in a sub cube. Figure shows a slice operation where the sales data are selected from the central cube for the dimension time using the criteria time = ‖Q2". The dice operation defines a sub cube by performing a selection on two or more dimensions.

**Pivot (rotate):** Pivot is a visualization operation which rotates the data axes in view in order to provide an alternative presentation of the data. Figure shows a pivot operation where the item and location axes in a 2-D slice are rotated.




## DATA WAREHOUSE ARCHITECTURE

To design an effective data warehouse we need to understand and analyze business needs and construct a business analysis framework.
Four different views regarding the design of a data warehouse must be considered: the top-down view, the data source view, the data warehouse view, and the business query view

The top-down view allows the selection of the relevant information necessary for the data warehouse. This information matches the current and future business needs.

The data source view exposes the information being captured, stored, and managed by operational systems. This information may be documented at various levels of detail and accuracy, from individual data source tables to integrated data source tables.

The data warehouse view includes fact tables and dimension tables. It represents the information that is stored inside the data warehouse, including pre calculated totals and counts, as well as information regarding the source, date, and time of origin.

Finally, the business query view is the perspective of data in the data warehouse from the viewpoint of the end user.

**The Process of Data Warehouse Design**

A data warehouse can be built using a top-down approach, a bottom-up approach, or a combination of both.

The top-down approach starts with the overall design and planning.

The bottom-up approach starts with experiments and prototypes.

In the combined approach, an organization can exploit the planned and strategic nature of the top-down approach while retaining the rapid implementation and opportunistic application of the bottom-up approach.

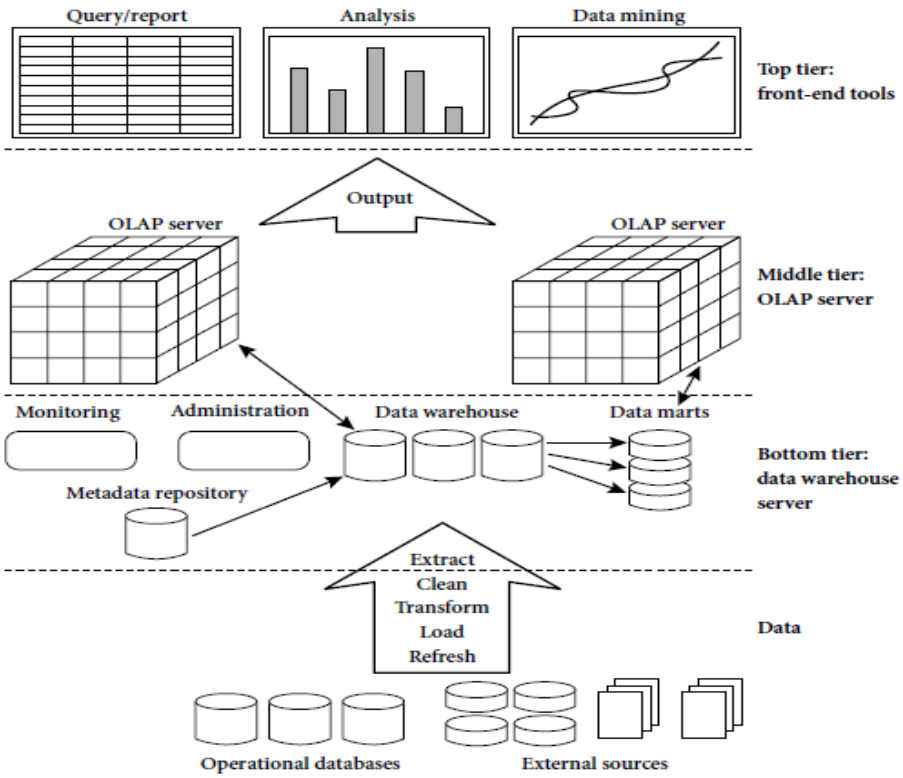
**From the software engineering point of view**, the design and construction of a data warehouse may consist of the following steps: planning, requirements study, problem analysis, warehouse design, data integration and testing, and finally deployment of the data warehouse.

Large software systems can be developed using two methodologies: the waterfall method or the spiral method.

The waterfall method performs a structured and systematic analysis at each step before proceeding to the next, which is like a waterfall, falling from one step to the next.

The spiral method involves the rapid generation of increasingly functional systems, with short intervals between successive releases.

**A Three-Tier Data Warehouse Architecture**




1.The bottom tier is a warehouse database server that is almost always a relational database system. Back-end tools and utilities are used to feed data into the bottom tier from operational databases or other external sources (such as customer profile information provided by external consultants). These tools and utilities perform data extraction, cleaning, and transformation (e.g., to merge similar data from different

2. The middle tier is an OLAP server that is typically implemented using either (1) a relational OLAP (ROLAP) model, that is, an extended relational DBMS that maps operations on multidimensional data to

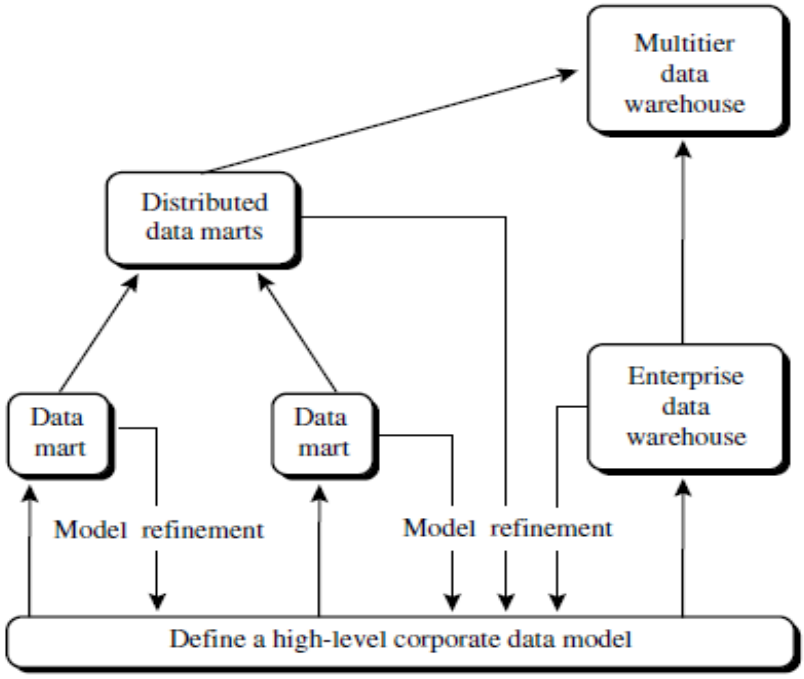
standard relational operations; or (2) a multidimensional OLAP (MOLAP) model, that is, a special-purpose server that directly implements multidimensional data and operations.

3.The top tier is a front-end client layer, which contains query and reporting tools, analysis tools, and/or data mining tools (e.g., trend analysis, prediction, and so on).

From the architecture point of view, there are three data warehouse models: the enterprise warehouse, the data mart, and the virtual warehouse.

1. Enterprise warehouse: An enterprise warehouse collects all of the information about subjects spanning the entire organization.
2. Data mart: A data mart contains a subset of corporate-wide data that is of value to a specific group of users. The scope is confined to specific selected subjects.
3. Virtual warehouse: A virtual warehouse is a set of views over operational databases. For efficient query processing, only some of the possible summary views may be materialized.

A recommended method for the development of data warehouse systems is to implement the warehouse in an incremental and evolutionary manner




**Data Warehouse Back-End Tools and Utilities**

Data extraction, which typically gathers data from multiple, heterogeneous, and external sources
Data cleaning, which detects errors in the data and rectifies them when possible
Data transformation, which converts data from legacy or host format to warehouse format
Load, which sorts, summarizes, consolidates, computes views, checks integrity, and builds indices and partitions
Refresh, which propagates the updates from the data sources to the warehouse

**Metadata Repository**

Metadata are data about data. When used in a data warehouse, metadata are the data that define

warehouse objects.
A metadata repository should contain the following:
A description of the structure of the data warehouse, which includes the warehouse schema, view, dimensions, hierarchies, and derived data definitions, as well as data mart locations and contents
Operational metadata, which include data lineage (history of migrated data and the sequence of transformations applied to it), currency of data (active, archived, or purged), and monitoring information (warehouse usage statistics, error reports, and audit trails)
The algorithms used for summarization, which include measure and dimension definition algorithms, data on granularity, partitions, subject areas, aggregation, summarization, and predefined queries and reports
The mapping from the operational environment to the data warehouse, which includes source databases and their contents, gateway descriptions, data partitions, data extraction, cleaning, transformation rules and defaults, data refresh and purging rules, and security (user authorization and access control)
Data related to system performance, which include indices and profiles that improve data access and retrieval performance, in addition to rules for the timing and scheduling of refresh, update, and replication cycles Business metadata, which include business terms and definitions, data ownership
information, and charging policies

**Types of OLAP Servers: ROLAP versus MOLAP versus HOLAP**

Relational OLAP (ROLAP) servers: They use a relational or extended-relational DBMS to store and manage warehouse data, and OLAP middleware to support missing pieces. ROLAP servers include optimization for each DBMS back end, implementation of aggregation navigation logic, and additional tools and services. ROLAP technology tends to have greater scalability than MOLAP technology.

Multidimensional OLAP (MOLAP) servers: These servers support multidimensional views of data through array-based multidimensional storage engines. They map multidimensional views directly to data cube array structures.

Hybrid OLAP (HOLAP) servers: The hybrid OLAP approach combines ROLAP and MOLAP technology, benefiting from the greater scalability of ROLAP and the faster computation of MOLAP.

Specialized SQL servers: To meet the growing demand of OLAP processing in relational databases, some database system vendors implement specialized SQL servers that provide advanced query language and query processing support for SQL queries over star and snowflake schemas in a read-only environment.

Mining Association Rules In Large Databases: Basic Concepts and a Road Map

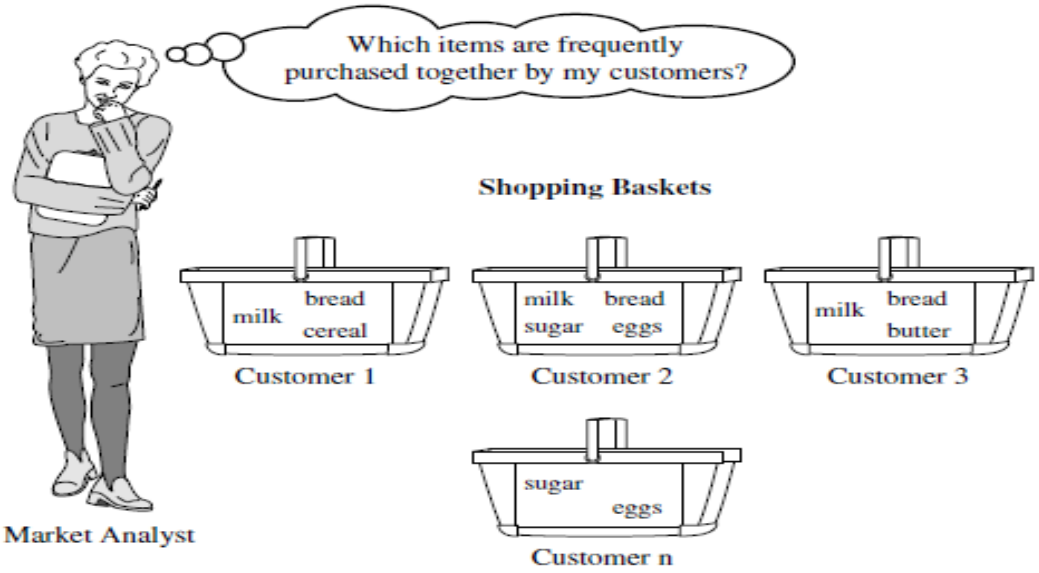
**Frequent patterns** are patterns (such as itemsets, subsequences, or substructures) that appear in a data set frequently.
For example, a set of items, such as milk and bread, that appear frequently together in a transaction data set is a frequent itemset.
A subsequence, such as buying first a PC, then a digital camera, and then a memory card, if it occurs frequently in a shopping history database, is a (frequent) sequential pattern.
A substructure can refer to different structural forms, such as subgraphs, subtrees, or sublattices, which may be combined with itemsets or subsequences. If a substructure occurs frequently, it is called a (frequent) structured pattern.
Finding such frequent patterns plays an essential role in mining associations, correlations, and many other interesting relationships among data.

How can we find frequent itemsets from large amounts of data, where the data are either transactional or

relational?
How can we mine association rules in multilevel and multidimensional space?
Which association rules are the most interesting?
How can we help or guide the mining procedure to discover interesting associations or correlations?
How can we take advantage of user preferences or constraints to speed up the mining process?

**Market Basket Analysis: A Motivating Example**

A typical example of frequent itemset mining is market basket analysis. This process analyzes customer buying habits by finding associations between the different items that customers place in their "shopping baskets".
The discovery of such associations can help retailers develop marketing strategies by gaining insight into which items are frequently purchased together by customers.
For instance, if customers are buying milk, how likely are they to also buy bread (and what kind of bread) on the same trip to the supermarket?
Such information can lead to increased sales by helping retailers do selective marketing and plan their shelf space.




If we think of the universe as the set of items available at the store, then each item has a boolean variable representing the presence or absence of that item.
Each basket can then be represented by a Boolean vector of values assigned to these variables.
The Boolean vectors can be analyzed for buying patterns that reflect items that are frequently *associated* or purchased together.
These patterns can be represented in the form of association rules. For example, the information that customers who purchase computers also tend to buy antivirus software at the same time is represented in Association Rule

*computer->antivirus software* [*support* = 2%; *confidence* = 60%]

**Support and Confidence measures**

Rule support and confidence are two measures of rule interestingness. They respectively reflect the usefulness and certainty of discovered rules.

A support of 2% for Association Rule means that 2% of all the transactions under analysis show that computer and antivirus software are purchased together.

A confidence of 60% means that 60% of the customers who purchased a computer also bought the software.

Typically, association rules are considered interesting if they satisfy both a minimum support threshold and a minimum confidence threshold. Such thresholds can be set by users or domain experts.

**Frequent Itemsets:**

Let I ={I1, I2, , .. , Im} be a set of items. Let D, the task-relevant data, be a set of database transactions where each transaction T is a set of items such that $T \subseteq I$ . Each transaction is associated with an identifier, called TID. Let A be a set of items. A transaction T is said to contain A if and only if $A \subseteq T$.

An association rule is an implication of the form $A \Rightarrow B$, where $A \subseteq I$ , $B \subseteq I$ , and $A \cap B = \emptyset$.

The rule $A \Rightarrow B$ holds in the transaction set D with support s, where s is the percentage of transactions in D that contain AUB (i.e., the union of sets A and B, or say, both A and B). This is taken to be the probability, P(AUB).

$$\textbf{support}(A \Rightarrow B) = P(A \cup B)$$

The rule $A \Rightarrow B$ has confidence c in the transaction set D, where c is the percentage of transactions in D containing A that also contain B. This is taken to be the conditional probability, P(B/A). That is,

$$\textbf{confidence}(A \Rightarrow B) = P(B|A)$$

A set of items is referred to as an itemset.

An itemset that contains k items is a k-itemset. The set {computer, antivirus software} is a 2-itemset. The occurrence frequency of an itemset is the number of transactions that contain the itemset. This is also known, simply, as the frequency, support count, or count of the itemset.

$$\textbf{confidence}(A \Rightarrow B) = P(B|A) = \textbf{support}(A \cup B) / \textbf{support}(A) = \textbf{support count}(A \cup B) / \textbf{support count}(A)$$

**Closed Itemset:**

An itemset X is closed in a data set S if there exists no proper super-itemset Y such that Y has the same support count as X in S. An itemset X is a closed frequent itemset in set S if X is both closed and frequent in S. An itemset X is a maximal frequent itemset (or max-itemset) in set S if X is frequent, and there exists no super-itemset Y such that $X \subseteq Y$ and Y is frequent in S.

**Association Rules**

In general, association rule mining can be viewed as a two-step process:

1. Find all frequent itemsets:

By definition, each of these itemsets will occur at least as frequently as a predetermined minimum support count, min sup.

2. Generate strong association rules from the frequent itemsets:

By definition, these rules must satisfy minimum support and minimum confidence

**FREQUENT ITEM SET GENERATION - THE APRIORI PRINCIPLE**

Apriori is a seminal algorithm proposed by R. Agrawal and R. Srikant in 1994 for mining frequent itemsets for Boolean association rules. The name of the algorithm is based on the fact that the algorithm uses prior knowledge of frequent itemset properties.

Apriori employs an iterative approach known as a level-wise search, where k-itemsets are used to explore

(k+1)-itemsets. First, the set of frequent 1-itemsets is found by scanning the database to accumulate the count for each item, and collecting those items that satisfy minimum support. The resulting set is denoted L1.
Next, L1 is used to find L2, the set of frequent 2-itemsets, which is used to find L3, and so on, until no more frequent k-itemsets can be found. The finding of each Lk requires one full scan of the database.
To improve the efficiency of the level-wise generation of frequent itemsets, an important property called the Apriori property

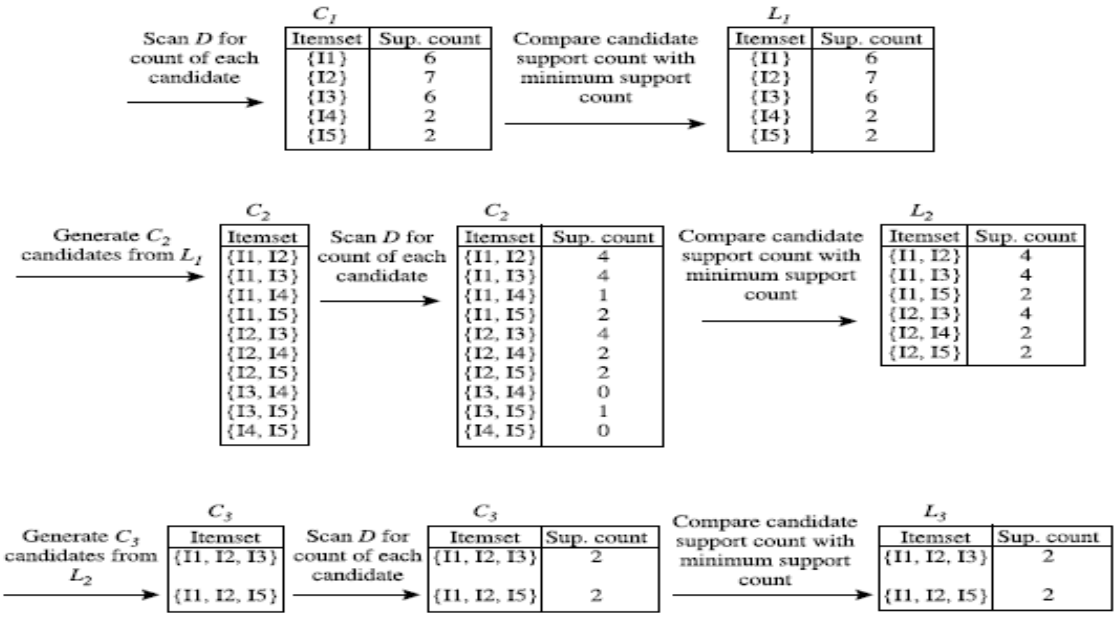
**Apriori property:** All nonempty subsets of a frequent itemset must also be frequent.
The Apriori property is based on the following observation. By definition, if an itemset I does not satisfy the minimum support threshold, min sup, then I is not frequent; that is, P(I) < min sup. If an item A is added to the itemset I, then the resulting itemset (i.e.,I U A) cannot occur more frequently than I. Therefore, I U A is not frequent either; that
is, P(I U A) < min sup.
This property belongs to a special category of properties called antimonotone in the sense that if a set cannot pass a test, all of its supersets will fail the same test as well. It is called antimonotone because the property is monotonic in the context of failing a test.

Transactional data for an *AllElectronics* branch.

| *TID* | *List of item_IDs* |
|---|---|
| T100 | I1, I2, I5 |
| T200 | I2, I4 |
| T300 | I2, I3 |
| T400 | I1, I2, I4 |
| T500 | I1, I3 |
| T600 | I2, I3 |
| T700 | I1, I3 |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3 |

Apriori. Let's look at a concrete example, based on the AllElectronics transaction database,D, of Table. There are nine transactions in this database, that is, |D| = 9.

1. In the first iteration of the algorithm, each item is a member of the set of candidate 1-itemsets, $C_1$. The algorithm simply scans all of the transactions in order to count the number of occurrences of each item.
2. Suppose that the minimum support count required is 2, that is, $min_sup = 2$. (Here, we are referring to *absolute* support because we are using a support count. The corresponding relative support is 2/9 = 22%). The set of frequent 1-itemsets, $L_1$, can then be determined. It consists of the candidate 1-itemsets satisfying minimum support. In our example, all of the candidates in $C_1$ satisfy minimum support.
3. To discover the set of frequent 2-itemsets, $L_2$, the algorithm uses the join $L_1 \bowtie L_1$ to generate a candidate set of 2-itemsets, $C_2$.[8] $C_2$ consists of $\binom{|L_1|}{2}$ 2-itemsets. Note that no candidates are removed from $C_2$ during the prune step because each subset of the candidates is also frequent.
4. Next, the transactions in $D$ are scanned and the support count of each candidate itemset in $C_2$ is accumulated, as shown in the middle table of the second row in Figure 5.2.
5. The set of frequent 2-itemsets, $L_2$, is then determined, consisting of those candidate 2-itemsets in $C_2$ having minimum support.
6. The generation of the set of candidate 3-itemsets, $C_3$, is detailed in Figure 5.3. From the join step, we first get $C_3 = L_2 \bowtie L_2 = \{\{I1, I2, I3\}, \{I1, I2, I5\}, \{I1, I3, I5\}, \{I2, I3, I4\}, \{I2, I3, I5\}, \{I2, I4, I5\}\}$. Based on the Apriori property that all subsets of a frequent itemset must also be frequent, we can determine that the four latter candidates cannot possibly be frequent. We therefore remove them from $C_3$, thereby saving the effort of unnecessarily obtaining their counts during the subsequent scan of $D$ to determine $L_3$. Note that when given a candidate $k$-itemset, we only need to check if its $(k-1)$-subsets are frequent since the Apriori algorithm uses a level-wise search strategy. The resulting pruned version of $C_3$ is shown in the first table of the bottom row of Figure 5.2.
7. The transactions in $D$ are scanned in order to determine $L_3$, consisting of those candidate 3-itemsets in $C_3$ having minimum support (Figure 5.2).
8. The algorithm uses $L_3 \bowtie L_3$ to generate a candidate set of 4-itemsets, $C_4$. Although the join results in $\{\{I1, I2, I3, I5\}\}$, this itemset is pruned because its subset $\{\{I2, I3, I5\}\}$ is not frequent. Thus, $C_4 = \phi$, and the algorithm terminates, having found all of the frequent itemsets. ■




**Generating Association Rules from Frequent Itemsets**

Once the frequent itemsets from transactions in a database D have been found,it is straightforward to generate strong association rules from them (where strong association rules satisfy both minimum support and minimum confidence).

**confidence(A⇒B) = P(B|A) = support count(A ∪B)/ support count(A)**

The conditional probability is expressed in terms of itemset support count, where support count(AUB) is the number of transactions containing the itemsets AUB, and support count(A) is the number of transactions containing the itemset A. Based on this equation, association rules can be generated as follows:

- For each frequent itemset $l$, generate all nonempty subsets of $l$.
- For every nonempty subset $s$ of $l$, output the rule "$s \Rightarrow (l - s)$" if $\frac{support_count(l)}{support_count(s)} \geq min_conf$, where $min_conf$ is the minimum confidence threshold.

Because the rules are generated from frequent itemsets, each one automatically satisfies minimum support. Frequent itemsets can be stored ahead of time in hash tables along with their counts so that they can be accessed quickly.

**Generating association rules.** Let's try an example based on the transactional data for *AllElectronics* shown in Table 5.1. Suppose the data contain the frequent itemset $l = \{I1, I2, I5\}$. What are the association rules that can be generated from $l$? The nonempty subsets of $l$ are $\{I1, I2\}$, $\{I1, I5\}$, $\{I2, I5\}$, $\{I1\}$, $\{I2\}$, and $\{I5\}$. The resulting association rules are as shown below, each listed with its confidence:

$$I1 \wedge I2 \Rightarrow I5, \quad confidence = 2/4 = 50\%$$
$$I1 \wedge I5 \Rightarrow I2, \quad confidence = 2/2 = 100\%$$
$$I2 \wedge I5 \Rightarrow I1, \quad confidence = 2/2 = 100\%$$
$$I1 \Rightarrow I2 \wedge I5, \quad confidence = 2/6 = 33\%$$
$$I2 \Rightarrow I1 \wedge I5, \quad confidence = 2/7 = 29\%$$
$$I5 \Rightarrow I1 \wedge I2, \quad confidence = 2/2 = 100\%$$

If the minimum confidence threshold is, say, 70%, then only the second, third, and last rules above are output, because these are the only ones generated that are strong. Note that, unlike conventional classification rules, association rules can contain more than one conjunct in the right-hand side of the rule. ■

**Algorithm: FP_growth.** Mine frequent itemsets using an FP-tree by pattern fragment growth.

**Input:**

- $D$, a transaction database;
- *min_sup*, the minimum support count threshold.

**Output:** The complete set of frequent patterns.

**Method:**

1. The FP-tree is constructed in the following steps:
   (a) Scan the transaction database $D$ once. Collect $F$, the set of frequent items, and their support counts. Sort $F$ in support count descending order as $L$, the *list* of frequent items.
   (b) Create the root of an FP-tree, and label it as "null." For each transaction *Trans* in $D$ do the following. Select and sort the frequent items in *Trans* according to the order of $L$. Let the sorted frequent item list in *Trans* be $[p|P]$, where $p$ is the first element and $P$ is the remaining list. Call **insert_tree**$([p|P], T)$, which is performed as follows. If $T$ has a child $N$ such that $N.item\text{-}name = p.item\text{-}name$, then increment $N$'s count by 1; else create a new node $N$, and let its count be 1, its parent link be linked to $T$, and its node-link to the nodes with the same *item-name* via the node-link structure. If $P$ is nonempty, call **insert_tree**$(P, N)$ recursively.

2. The FP-tree is mined by calling **FP_growth**(*FP_tree*, *null*), which is implemented as follows.

```
procedure FP_growth(Tree, α)
(1)  if Tree contains a single path P then
(2)      for each combination (denoted as β) of the nodes in the path P
(3)          generate pattern β∪α with support_count = minimum support count of nodes in β;
(4)  else for each a_i in the header of Tree {
(5)      generate pattern β = a_i ∪ α with support_count = a_i.support_count;
(6)      construct β's conditional pattern base and then β's conditional FP_tree Tree_β;
(7)      if Tree_β ≠ ∅ then
(8)          call FP_growth(Tree_β, β); }
```

**Mining Frequent Itemsets without Candidate Generation**

Transactional data for an *AllElectronics* branch.

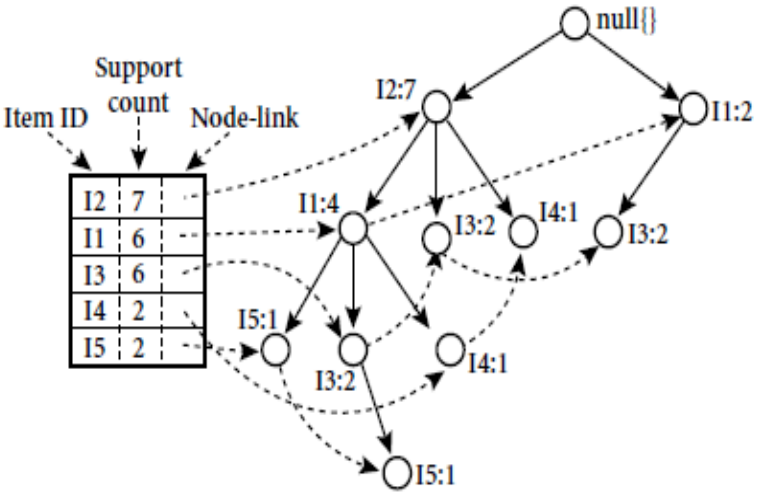
| TID | List of item_IDs |
|---|---|
| T100 | I1, I2, I5 |
| T200 | I2, I4 |
| T300 | I2, I3 |
| T400 | I1, I2, I4 |
| T500 | I1, I3 |
| T600 | I2, I3 |
| T700 | I1, I3 |
| T800 | I1, I2, I3, I5 |
| T900 | I1, I2, I3 |

The first scan of the database is the same as Apriori, which derives the set of frequent items (1-itemsets) and their support counts (frequencies). Let the minimum support count be 2. The set of frequent items is sorted in the order of descending support count. This resulting set or *list* is denoted *L*. Thus, we have *L* ={{I2: 7}, {I1: 6}, {I3: 6},{I4: 2}, {I5: 2}}.

An FP-tree is then constructed as follows. First, create the root of the tree, labeled with "null." Scan database *D* a second time. The items in each transaction are processed in *L* order (i.e., sorted according to descending support count), and a branch is created for each transaction. For example, the scan of the first transaction, "T100: I1, I2, I5," which contains three items (I2, I1, I5 in *L* order), leads to the construction of the first branch of the tree with three nodes, <I2: 1>, <I1:1>, and <I5: 1>, where I2 is linked as a child of the root, I1 is linked to I2, and I5 is linked to I1.

The second transaction, T200, contains the items I2 and I4 in *L* order, which would result in a branch where I2 is linked to the root and I4 is linked to I2. However, this branch would share a common prefix, I2, with the existing path for T100.

Therefore, we instead increment the count of the I2 node by 1, and create a new node, <I4: 1>,which is linked as a child of <I2: 2>. In general, when considering the branch to be added for a transaction, the count of each node along a common prefix is incremented by 1, and nodes for the items following the prefix are created and linked accordingly. To facilitate tree traversal, an item header table is built so that each item points to its occurrences in the tree via a chain of node-links. The tree obtained after scanning all of the transactions is shown in Figure with the associated node-links. In this way, the problem of mining frequent patterns in databases is transformed to that of mining the FP-tree.




Mining the FP-tree by creating conditional (sub-)pattern bases.

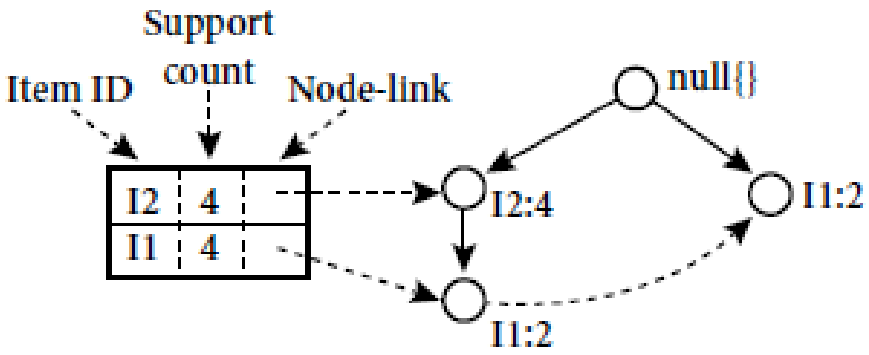
| Item | Conditional Pattern Base | Conditional FP-tree | Frequent Patterns Generated |
|---|---|---|---|
| I5 | {{I2, I1: 1}, {I2, I1, I3: 1}} | ⟨I2: 2, I1: 2⟩ | {I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2} |
| I4 | {{I2, I1: 1}, {I2: 1}} | ⟨I2: 2⟩ | {I2, I4: 2} |
| I3 | {{I2, I1: 2}, {I2: 2}, {I1: 2}} | ⟨I2: 4, I1: 2⟩, ⟨I1: 2⟩ | {I2, I3: 4}, {I1, I3: 4}, {I2, I1, I3: 2} |
| I1 | {{I2: 4}} | ⟨I2: 4⟩ | {I2, I1: 4} |

The FP-tree is mined as follows. Start from each frequent length-1 pattern (as an initial suffix pattern), construct its conditional pattern base (a "sub database,"which consists of the set of prefix paths in the FP-tree co-occurring with the suffix pattern), then construct its (conditional) FP-tree, and perform mining recursively on such a tree. The pattern growth is achieved by the concatenation of the suffix pattern with the frequent patterns generated from a conditional FP-tree.

Mining of the FP-tree is summarized in Table and detailed as follows. We first consider I5, which is the last item in *L*, rather than the first. The reason for starting at the end of the list will become apparent as we explain the FP-tree mining process. I5 occurs in two branches of the FP-tree of Figure. (The occurrences of I5 can easily be found by following its chain of node-links.) The paths formed by these branches are <I2, I1,I5: 1> and <I2, I1, I3, I5: 1>. Therefore, considering I5 as a suffix, its corresponding two prefix paths are <I2, I1: 1> and <I2, I1, I3: 1>, which form its conditional pattern base. Its conditional FP-tree contains only a single path,<I2: 2, I1: 2>; I3 is not included because its support count of 1 is less than the minimum support count. The single path generates all the combinations of frequent patterns: {I2, I5: 2}, {I1, I5: 2}, {I2, I1, I5: 2}. For I4, its two prefix paths form the conditional pattern base, {{I2 I1: 1}, {I2: 1}},which generates a single-node conditional FP-tree, <I2: 2>, and derives one frequent

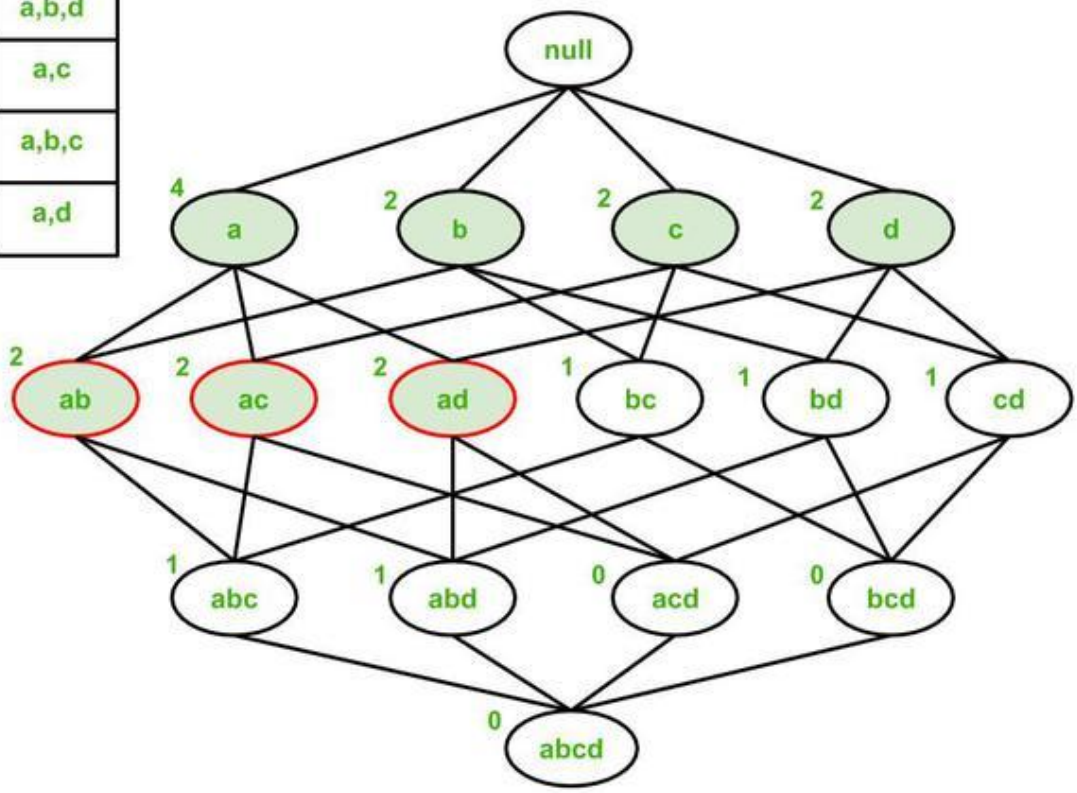
pattern, {I2, I1: 2}. Notice that although I5 follows I4 in the first branch, there is no need to include I5 in the analysis here because any frequent pattern involving I5 is analyzed in the examination of I5.
Similar to the above analysis, I3's conditional pattern base is {{I2, I1: 2}, {I2: 2},{I1: 2}}. Its conditional FP-tree has two branches, <I2: 4, I1: 2> and <I1: 2>, as shown in Figure which generates the set of patterns, {{I2, I3: 4}, {I1, I3: 4}, {I2, I1, I3: 2}}.

Finally, I1's conditional pattern base is {{I2: 4}}, whose FP-tree contains only one node, <I2: 4>, which generates one frequent pattern, {I2, I1: 4}. This mining process is summarized in Figure.




**Compact frequent itemsets-Maximal frequent itemset** :
The number of frequent itemsets generated by the Apriori algorithm can often be very large, so it is beneficial to identify a small representative set from which every frequent itemset can be derived. One such approach is using maximal frequent itemsets.
A maximal frequent itemset is a frequent itemset for which none of its immediate supersets are frequent. To illustrate this concept, consider the example given below:

| TID | Items |
|---|---|
| 1 | a,b,d |
| 2 | a,c |
| 3 | a,b,c |
| 4 | a,d |




The support counts are shown on the top left of each node. Assume support count threshold = 50%, that is, each item must occur in 2 or more transactions. Based on that threshold, the frequent itemsets are: a, b, c, d, ab, ac and ad (shaded nodes).
Out of these 7 frequent itemsets, 3 are identified as maximal frequent (having red outline):
ab: Immediate supersets abc and abd are infrequent.
ac: Immediate supersets abc and acd are infrequent.
ad: Immediate supersets abd and acd are infrequent.
The remaining 4 frequent nodes (a, b, c and d) cannot be maximal frequent because they all have at least 1 immediate superset that is frequent.

**Closed frequent itemset:**
An itemset is closed if none of its immediate superset has the same support as that of the itemset.